# RewaCiti

## Codebase Walkthrough

Every section shows the actual source code and then explains what it does. Read top to bottom: each file is presented in the order the app runs it.

| | |
|---|---|
| **Frontend** | React 18 · TypeScript · Vite (rolldown) · Tailwind v4 · Zustand · Axios |
| **Backend / services** | SabiFlow API · Paystack Inline JS · OpenStreetMap (Leaflet) |

# Contents

**1. How the app boots** **5**
`src/app/main.tsx` — entry point . . . 5
`src/app/App.tsx` — providers, theme sync, wishlist hydration . . . 5

**2. Types — the vocabulary of the whole app** **7**
The app shape: `Property` . . . 7
The wire shape: `SabiFlowProduct` . . . 8
Store contracts . . . 9
Other shapes . . . 9

**3. Shared layer: utils, cookies, theme, images** **10**
`src/shared/lib/utils.ts` — the whole shared toolbox (63 lines) . . . 10
`src/shared/store/useThemeStore.ts` — light/dark mode . . . 11
`src/shared/hooks/useScrollToHash.ts` — deep-link scrolling . . . 11
`src/shared/components/ui/LazyImage.tsx` — lazy loading with fallback . . . 12
`src/shared/components/ui/Skeleton.tsx` + `Skeletons.tsx` . . . 13

**4. Auth: store, API, and the 5 pages** **14**
`src/features/auth/store/useAuthStore.ts` — the auth brain (144 lines) . . . 14
`src/features/auth/services/authAPI.ts` — every auth endpoint (260 lines) . . . 15
`src/features/auth/pages/Login.tsx` — login *and* sign-up in one page . . . 17
`src/features/auth/pages/VerifyEmail.tsx` — 6-box OTP (308 lines) . . . 18
`src/features/auth/pages/ForgotPassword.tsx` — two-step wizard (326 lines) . . . 19
`src/features/auth/pages/ResetPassword.tsx` (213 lines) . . . 19
`src/features/auth/pages/Profile.tsx` — two tabs (747 lines) . . . 20
`src/features/auth/components/ProfileDropdown.tsx` (76 lines) . . . 21

**5. Properties: the store (the core of the app)** **22**
5.1 The API client and small helpers . . . 22
5.2 The mapper — API shape → app shape . . . 23
5.3 Wishlist metadata cache . . . 25
5.4 Fetching the list . . . 25
5.5 Other read actions . . . 26
5.6 Related properties . . . 27
5.7 Detail page: cache-first loading . . . 27
5.8 Pagination . . . 28
5.9 Client-side search . . . 29
5.10 Wishlist: fetch + optimistic toggle . . . 29
5.11 Legacy client-side filter . . . 31

**6. Properties: listing page** **32**
6.1 State . . . 32
6.2 Measuring the sticky bar (for the map offset) . . . 32
6.3 iOS-safe body scroll lock . . . 33
6.4 The one effect that fetches everything . . . 33
6.5 Two-way sync between the stepper and the bedroom filter . . . 34
6.6 Price text → numbers . . . 34

6.7 Filter option builders . . . 34
6.8 Cascading location logic . . . 35
6.9 Active filter badge + clear all . . . 36
6.10 List ↔ Map mode . . . 36
6.11 The custom-fields service . . . 37

**7. Properties: detail page** **39**
7.1 Getting the property (cache-first, three sources) . . . 39
7.2 State . . . 39
7.3 Image carousel . . . 40
7.4 Sharing . . . 41
7.5 Shortlist . . . 41
7.6 Video tour . . . 42
7.7 The sticky bottom action bar . . . 42
7.8 Body scroll lock while a modal is open . . . 43
7.9 Related properties . . . 43
7.10 The modals it owns . . . 43

**8. Properties: shared components** **44**
8.1 `PropertyCard.tsx` . . . 44
8.2 `propertyFeatureIcons.ts` . . . 46
8.3 `PropertyFeatureCard.tsx` / `PropertyFeatureItem.tsx` . . . 48
8.4 `FeatureSection.tsx` . . . 49
8.5 `CustomDropdown.tsx` . . . 49
8.6 `PropertyMap.tsx` . . . 50
8.7 `PropertySection.tsx` . . . 51

**9. Properties: modals** **53**
9.1 The shared pattern . . . 53
9.2 `PropertyPaymentModal.tsx` — buying a property . . . 54
9.3 `ServiceRatingModal.tsx` — rating + Trustpilot . . . 56
9.4 `ReportAgentModal.tsx` — fallback endpoints . . . 57
9.5 `PreferenceModal.tsx` — property search brief . . . 58
9.6 `CoRentModal.tsx` — co-tenant matching . . . 58
9.7 `ListPropertyModal.tsx` — become a lister . . . 59
9.8 `PropertyFiltersModal.tsx` — hand-rolled filter sheet . . . 59

**10. Inspections (book a visit)** **60**
10.1 `useInspectionStore.ts` . . . 60
10.2 `BookInspectionModal.tsx` . . . 60

**11. Blog, Comments, FAQ and the area-map store** **64**
11.1 `useBlogStore.ts` . . . 64
11.2 `BlogPage.tsx` — listing . . . 65
11.3 `BlogPostDetailsPage.tsx` — single post . . . 66
11.4 `CommentSection.tsx` — testimonials . . . 67
11.5 `AllCommentsPage.tsx` / `AllFAQPage.tsx` . . . 67
11.6 `FAQSection.tsx` — home-page FAQ strip . . . 68
11.7 `useAreaMapStore.ts` — currently unused . . . 68

**12. Layout: Navbar and Footer** **69**
12.1 `Navbar.tsx` . . . 69
12.2 `Footer.tsx` . . . 71

**13. Marketing and legal pages** **73**
13.1 `Home.tsx` . . . 73
13.2 `About.tsx` and `Service.tsx` . . . 74
13.3 `Contact.tsx` — the only marketing page with a form . . . 74
13.4 `Terms.tsx` / `PrivacyPolicy.tsx` . . . 75
13.5 `NotFound.tsx` . . . 75

**14. Build, config and deployment** **76**
14.1 `package.json` . . . 76
14.2 `vite.config.ts` . . . 77
14.3 `tsconfig.app.json` . . . 77
14.4 `eslint.config.js` . . . 78
14.5 Tailwind / CSS . . . 78
14.6 `index.html` — Trustpilot bootstrap . . . 79
14.7 `vercel.json` — SPA routing . . . 80
14.8 `.env` — environment variables . . . 80
14.9 Verified commands . . . 80

**15. End-to-end data flows** **81**
15.1 Every backend endpoint the app uses . . . 81
15.2 App boot . . . 82
15.3 Auth . . . 82
15.4 Browsing properties . . . 82
15.5 Wishlist / shortlist . . . 83
15.6 Buying a property . . . 83
15.7 Booking a visit . . . 83
15.8 Lead capture (CRM) . . . 83
15.9 Static content . . . 84
15.10 Where state lives . . . 84

# 1. How the app boots

## `src/app/main.tsx` — entry point

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './styles/App.css'
import App from './App.tsx'
import "leaflet/dist/leaflet.css";

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

### How it works

- `index.html` contains a single `<div id="root">`; React replaces its contents.
- `App.css` is the Tailwind entry (imported first so utilities are available).
- `leaflet/dist/leaflet.css` is imported globally so map styles exist before any map renders.
- `StrictMode` double-invokes effects in dev — useful for catching cleanup bugs.

## `src/app/App.tsx` — providers, theme sync, wishlist hydration

```
function App() {
  const theme = useThemeStore((s) => s.theme);
  const { token, isAuthenticated } = useAuthStore();

  // 1. Theme sync -> toggles the `dark` class on <html>
  useEffect(() => {
    if (theme === "dark") {
      document.documentElement.classList.add("dark");
    } else {
      document.documentElement.classList.remove("dark");
    }
  }, [theme]);

  // 2. Wishlist hydration -> keeps the navbar badge correct after a reload
  useEffect(() => {
    if (isAuthenticated && token) {
      void usePropertyStore.getState().fetchWishlist();
    }
  }, [isAuthenticated, token]);

  return (
    <HelmetProvider>
      <BrowserRouter>
        <Toaster
          theme={theme as "light" | "dark"}
          closeButton
          position="top-right"
          toastOptions={{
            className: "font-sans",
            style: { borderRadius: '0.625rem' },
            classNames: {
              toast: "bg-white dark:bg-[#1A1A1A] text-gray-900 dark:text-white border-purple-100 dark:border-gray-800
↪  shadow-xl border",
              description: "text-gray-500 dark:text-gray-400",
              actionButton: "bg-[#703BF7] text-white hover:bg-[#9677df] transition-colors",
```

```
            cancelButton: "bg-gray-100 dark:bg-white/10 ...",
            success: "text-[#0F766E] dark:text-[#5EEAD4]",
            error: "text-[#B91C1C] dark:text-[#FCA5A5]",
            warning: "text-[#D97706] dark:text-[#FBBF24]",
            info: "text-[#703BF7] dark:text-[#9677df]",
          }
        }}
      />
      <Routes>
        <Route index path="/" element={<Home />} />
        <Route path="/auth/login" element={<Login />} />
        <Route path="/auth/verify-email" element={<VerifyEmail />} />
        <Route path="/auth/forgot-password" element={<ForgotPassword />} />
        <Route path="/auth/reset-password" element={<ResetPassword />} />
        <Route path="/auth/profile" element={<Profile />} />
        <Route path="/AllComments" element={<AllComments />} />
        <Route path="/AllFAQs" element={<AllFAQs />} />
        <Route path="/About" element={<About />} />
        <Route path="/properties" element={<Properties />} />
        <Route path="/properties/:slug" element={<PropertyDetails />} />
        <Route path="/property-feature-icons" element={<PropertyFeatureIconsPage />} />
        <Route path="/Service" element={<Service />} />
        <Route path="/Contact" element={<Contact />} />
        <Route path="/Studentarea" element={<Navigate to="/properties" replace />} />
        <Route path="/terms" element={<Terms />} />
        <Route path="/privacy-policy" element={<PrivacyPolicy />} />
        <Route path="/blog" element={<BlogPage />} />
        <Route path="/blog/:slug" element={<BlogPostDetailsPage />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  </HelmetProvider>
 );
}
```

**How it works**

- **Provider order** matters: `HelmetProvider` → `BrowserRouter` → `Toaster` + `Routes`.
- **Theme**: the `dark` class on `<html>` is the single switch for the whole app. Tailwind v4 is configured with `@custom-variant dark (&:is(.dark *))`, so every `dark:` utility only applies when that class is present.
- **Wishlist hydration**: `fetchWishlist()` is called directly on the store (`getState()`) rather than through a hook, because this is not a component.
- **No route guard.** `/auth/profile` is publicly routable — pages check `useAuthStore.getState().isAuthenticated` themselves.
- `Toaster` is themed with the same brand colours as the rest of the app.

# 2. Types — the vocabulary of the whole app

Everything lives in one file: `src/types/index.ts` (358 lines).

### The app shape: `Property`

```ts
export interface Property {
  id: string;
  sku?: string;
  slug: string;
  name: string;
  img: string;
  images: string[];
  description: string;
  bedrooms: number ;
  bathrooms: number | string;
  category: string;
  duration?: string;
  listingType?: string;
  pricing: {
   LegalFee: number;
   ServiceFee: number;
   CautionFee: number;
   PropertyCost: number;
   AgentFee:number;
   TotalCost: number;
  };
  createdBy?: {
    _id: string;
    id: string;
    firstName: string;
    lastName: string;
    email?: string;
    classification: string | null;
  } | null;
  creatorClassification?: string | null;
  location: {
    area: string;
    city: string;
    city_town?: string;
    state: string;
    nearest_university?: string;
  };
  geo_location: { lat: number; lng: number; address?: string; };
  yearBuilt: number;
  keyFeatures: string[];
  additionalFeatures?: string[];
  billsAndCharges?: string[];
  additionalBillsAndCharges?: string[];
  rules?: string[];
  additionalRules?: string[];
  specialNotes?: string[];
  specifications?: { label: string; value: string }[];
  videoUrl?: string;
  caretakerContact?: { whatsapp?: string; phone?: string; };
  visitationfee: number;
}
```

### The wire shape: `SabiFlowProduct`

This is what the inventory API actually returns — **snake_case, nested under `customData`**:

```ts
export interface SabiFlowProduct {
  _id: string;
  sku?: string;
  name: string;
  slug: string;
  description: string;
  price: number;
  images: string[];
  thumbnail: string;
  videoUrl?: string;
  specifications?: Record<string, string>;
  categoryId?: { _id?: string; name: string; id?: string; };
  creatorClassification?: string;
  createdBy?: {
    _id: string; firstName: string; lastName: string; id: string;
    email?: string; classification?: string;
  } | string | null;
  customData?: {
    bedrooms?: number;
    bathrooms?: number|string;
    yearBuilt?: number;
    duration?: string;
    rules?: string[] | {
      key_rules?: string[];
      additional_rules?: string[];
    };
    pricing?: {
      agent_fee?: number;
      legal_fee?: number;
      service_fee?: number;
      caution_fee?: number;
      property_cost?: number;
    };
    key_features_and_amenities?: string[] | {
      other_features_and_amenities?: string[];
    };
    features_and_amenities?: {
      key_features_and_amenities?: string[];
      additional_features_and_amenities?: string[];
    };
    bills_and_charges?: {
      baic_bills_and_charges?: string[];
      basic_bills_and_charges?: string[];
      additional_bills_and_charges?: string[];
    };
    location?: {
      area?: string; city?: string; city_town?: string;
      state?: string; nearest_university?: string;
    };
    geo_location?: { lat: number; lng: number; address?: string; };
    care_taker_contact_optional?: {
      wattsapp_contact?: string;   // note: API misspells "whatsapp"
      call_contact?: string;
    };
    visitation_fee: number;
    special_notes?: string[];
    specifications?: { label: string; value: string }[];
    listing_type?: string;
  };
}
```

**Why two shapes?** The API is generic (it can sell anything), so listing data is buried in `customData` with snake_case keys. The UI wants a flat, camelCase-friendly object. `mapSabiFlowProductsToProperties()` (see §5) is the

translator between them — it is the most important function in the codebase.

## Store contracts

Each Zustand store has an interface in the same file, e.g.:

```ts
export interface PropertyStore {
  properties: Property[];
  filteredProperties: Property[];
  loading: boolean;
  error: string | null;
  fees: PropertyPaymentFees | null;

  ITEMS_PER_PAGE: number;
  page: number;        // 0-based, for the UI
  apiPage: number;     // 1-based, for the API
  totalProperties: number;
  filters: Record<string, string | number | undefined>;

  fetchProperties: (apiPage?: number, filters?: Record<string, string | number | undefined>) => Promise<void>;
  fetchPropertyBySlug: (slug: string) => Promise<void>;
  toggleShortlist: (property: Property) => Promise<void>;
  // ...
}
```

**Note the two page counters** — `page` is 0-based for rendering the pager, `apiPage` is 1-based for the `?page=` query param.

## Other shapes

```ts
export interface Comment { id: number; rating: number; summary: string;
  description: string; img: string; name: string; location: string; }

export interface FAQ { id: number; question: string; answer: string; }

export interface BlogPost { id: string; title: string; slug: string; content: string;
  summary: string; author: string; date: string; image: string; category: string; tags: string[]; }

export interface AreaMap { id: string; name: string; city: string;
  city_town?: string; state: string; areas: string[]; }

export interface Inspection {
  id: string; propertyId: string; propertyName: string;
  userName: string; userEmail: string; userPhone: string;
  date: string; time: string;
  status: "pending" | "confirmed" | "completed" | "cancelled";
  paymentStatus: "pending" | "paid" | "failed";
  amount: number; reference: string; location: string;
}

export interface Category { _id: string; id: string; name: string; slug: string; }

export interface InventoryFilters {
  creators: { id: string; firstName: string; lastName: string; classification: string; }[];
  vendorTracking: { enabled: boolean; classifications: string[]; };
  customFields: {
    key: string; name: string; type: string;
    options: string[] | null; queryParam: string; operators: string[];
  }[];
}
```

# 3. Shared layer: utils, cookies, theme, images

### `src/shared/lib/utils.ts` — the whole shared toolbox (63 lines)

```ts
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export const formatCurrency = (amount: number) => {
  if (amount >= 1000000000) {
    return `${(amount / 1000000000).toFixed(3).replace(/\.?0+$/, "")}B`;
  }
  if (amount >= 1000000) {
    return `${(amount / 1000000).toFixed(3).replace(/\.?0+$/, "")}M`;
  }
  return amount.toLocaleString();
};

export const ensureHttps = (url: string) => {
  if (!url) return url;
  if (url.startsWith("http://")) {
    return url.replace("http://", "https://");
  }
  return url;
};

export const stripHtmlTags = (html: string = ""): string => {
  if (!html) return "";
  return html
    .replace(/<br\s*\/?>/gi, "\n")
    .replace(/<\/(p|div|h[1-6]|li|tr)>/gi, "\n")
    .replace(/<[^>]+>/g, " ")
    .replace(/ /gi, " ")
    .replace(/&amp;/gi, "&")
    .replace(/&lt;/gi, "<")
    .replace(/&gt;/gi, ">")
    .replace(/&quot;/gi, '"')
    .replace(/&#39;/gi, "'")
    .replace(/[ \t]+/g, " ")
    .replace(/\n\s*\n+/g, "\n\n")
    .trim();
};

export const setCookie = (name: string, value: string, days = 365) => {
  const expires = new Date();
  expires.setTime(expires.getTime() + days * 24 * 60 * 60 * 1000);
  const secureFlag = window.location.protocol === "https:" ? ";Secure" : "";
  document.cookie = `${name}=${encodeURIComponent(value)};expires=${expires.toUTCString()};path=/;Sa
  ↪  meSite=Lax${secureFlag}`;
};

export const getCookie = (name: string) => {
  if (typeof document === "undefined") return "";
  const match = document.cookie.match(new RegExp(`(?:^|; )${name}=([^;]*)`));
  return match ? decodeURIComponent(match[1]) : "";
};

export const deleteCookie = (name: string) => {
  document.cookie = `${name}=;expires=Thu, 01 Jan 1970 00:00:00 GMT;path=/`;
};
```

#### How it works

- `cn()` = `twMerge(clsx(...))`. `clsx` handles conditional class names, `tailwind-merge` resolves Tailwind conflicts (later class wins) — e.g. `cn("p-2", "p-4")` → `"p-4"`.

- `formatCurrency` shows compact Naira: `1500000` → `"1.5M"`, `2500000000` → `"2.5B"`, `450000` → `"450,000"`. The `.replace(/\{}.?0+$/, "")` trims trailing zeros (`2.000M` → `2M`).
- `ensureHttps` fixes CDN images that are served over plain HTTP (mixed-content blocks them).
- `stripHtmlTags` converts API rich text into clean plain text for copy/share.
- **Cookies**: `path=/` so every route can read them, `SameSite=Lax`, and `Secure` automatically when served over HTTPS. 365-day default lifetime.

## `src/shared/store/useThemeStore.ts` — light/dark mode

```ts
export const useThemeStore = create<ThemeState>((set) => ({
  theme: (localStorage.getItem("theme") as Theme) || "light",

  toggleTheme: () =>
    set((state) => {
      const newTheme = state.theme === "light" ? "dark" : "light";
      localStorage.setItem("theme", newTheme);
      return { theme: newTheme };
    }),

  setTheme: (theme) => {
    localStorage.setItem("theme", theme);
    set({ theme });
  },
}));
```

### How it works

- The store reads `localStorage` at module-load time (so there's no flash of the wrong theme) and writes on every change.
- It does **not** touch the DOM. `App.tsx`'s effect watches `theme` and toggles the `dark` class on `<html>` — that is the only place the class is applied.

## `src/shared/hooks/useScrollToHash.ts` — deep-link scrolling

```ts
const useScrollToHash = (): void => {
  const location = useLocation();

  useEffect(() => {
    if (!location.hash) return;
    const id = location.hash.replace("#", "");
    const element = document.getElementById(id) as HTMLElement | null;
    if (element) {
      element.scrollIntoView({ behavior: "smooth", block: "start" });
    }
  }, [location]);
};
```

Used by `Home`, `About`, `Service`, `Contact` and `PropertiesPage` so footer links like `/#features` or `/About#Journey` land on the right section.

## `src/shared/components/ui/LazyImage.tsx` — lazy loading with fallback

```
export const LazyImage: React.FC<LazyImageProps> = ({
  src, alt, className = "", containerClassName = "", placeholderClassName = "",
  fallbackSrc = "/logo/Image.png", threshold = 0.01, rootMargin = "250px",
  onLoad, onError, ...props
}) => {
  const [isInView, setIsInView] = useState(false);
  const [isLoaded, setIsLoaded] = useState(false);
  const [hasError, setHasError] = useState(false);
  const containerRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    setIsLoaded(false);
    setHasError(false);
  }, [src]);                        // reset when src changes

  useEffect(() => {
    if (typeof window === "undefined" || !("IntersectionObserver" in window)) {
      setIsInView(true);            // graceful degradation
      return;
    }
    const observer = new IntersectionObserver(
      ([entry]) => {
        if (entry.isIntersecting) {
          setIsInView(true);
          observer.disconnect();    // one-shot: never unobserve again
        }
      },
      { threshold, rootMargin }
    );
    if (containerRef.current) observer.observe(containerRef.current);
    return () => observer.disconnect();
  }, [threshold, rootMargin]);

  const imageSource = hasError && fallbackSrc ? fallbackSrc : src;

  return (
    <div ref={containerRef} className={`relative overflow-hidden ${containerClassName}`}>
      {!isLoaded && (
        <div className={`absolute inset-0 bg-gray-200 dark:bg-neutral-800 animate-pulse z-0 ${placeholderClassName}`} />
      )}
      {isInView && (
        <img
          src={imageSource} alt={alt} loading="lazy" decoding="async"
          onLoad={() => setIsLoaded(true)}
          onError={() => { setHasError(true); setIsLoaded(true); }}
          className={`transition-opacity duration-300 ease-in-out ${isLoaded ? "opacity-100" : "opacity-0"} ${className}`}
          {...props}
        />
      )}
    </div>
  );
};
```

### How it works

- The `<img>` is **not rendered at all** until the container is within `250px` of the viewport (`rootMargin`) — so the browser never requests off-screen images.
- A pulsing grey block stands in until the image decodes.
- On load, opacity animates `0 → 1` over 300 ms (no layout shift).
- On error, the src is swapped to `fallbackSrc` (`/logo/Image.png`).

### `src/shared/components/ui/Skeleton.tsx` + `Skeletons.tsx`

```
function Skeleton({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) {
  return (
    <div className={cn("animate-pulse rounded-md bg-black/10 dark:bg-white/10", className)} {...props} />
  );
}
```

`Skeletons.tsx` composes that primitive into layout-matching placeholders:

| Component | Mirrors |
|---|---|
| `PropertyCardSkeleton` | a `PropertyCard` (image, title, location, description, bed/bath pills, price + button) |
| `CommentCardSkeleton` | a testimonial card |
| `FAQCardSkeleton` | an FAQ card |
| `PropertyDetailsSkeleton` | the entire detail page: header, gallery, video, two-column body, related grid, bottom action bar |
| `ProfileDropdownSkeleton` | the profile dropdown |
| `ProfilePageSkeleton` | the profile page two-column layout |

**Why it matters:** the skeleton markup mirrors the real DOM structure, so when the data arrives the layout does not jump.

# 4. Auth: store, API, and the 5 pages

### `src/features/auth/store/useAuthStore.ts` — the auth brain (144 lines)

```
export const COMPANY_ID = import.meta.env.VITE_COMPANY_ID;
const AUTH_TOKEN_COOKIE = "rewaciti_auth_token";
const AUTH_CUSTOMER_COOKIE = "rewaciti_auth_customer";

const getCookieToken = () => {
  if (typeof document === "undefined") return null;
  const token = getCookie(AUTH_TOKEN_COOKIE);
  return token || null;
};

const getCookieCustomer = (): Customer | null => {
  if (typeof document === "undefined") return null;
  const customerCookie = getCookie(AUTH_CUSTOMER_COOKIE);
  if (!customerCookie) return null;
  try {
    return JSON.parse(customerCookie) as Customer;
  } catch {
    return null;
  }
};

// Read cookies at module-load time so the very first render is already authed.
const initialToken = getCookieToken();
const initialCustomer = getCookieCustomer();

export const useAuthStore = create<AuthStore>()(
  persist(
    (set) => ({
      token: initialToken,
      customer: initialCustomer,
      isAuthenticated: !!initialToken || !!initialCustomer,
      companyId: COMPANY_ID,

      setToken: (token) => {
        if (token) {
          setCookie(AUTH_TOKEN_COOKIE, token);
        } else {
          deleteCookie(AUTH_TOKEN_COOKIE);
        }
        set({ token, isAuthenticated: !!token });
      },

      setCustomer: (customer) => {
        if (customer) {
          setCookie(AUTH_CUSTOMER_COOKIE, JSON.stringify(customer));
        } else {
          deleteCookie(AUTH_CUSTOMER_COOKIE);
        }
        set({ customer, isAuthenticated: !!customer });
      },

      setCompanyId: (companyId) => set({ companyId }),

      logout: () => {
        deleteCookie(AUTH_TOKEN_COOKIE);
        deleteCookie(AUTH_CUSTOMER_COOKIE);
        set({ token: null, customer: null, isAuthenticated: false });
      },
    }),
    {
      name: "auth-storage",
```

```
      onRehydrateStorage: () => (state) => {
        if (!state) return;
        const cookieToken = getCookieToken();
        const cookieCustomer = getCookieCustomer();
        if (cookieToken || cookieCustomer) {
          state.token = cookieToken;
          state.customer = cookieCustomer;
          state.isAuthenticated = !!(cookieToken || cookieCustomer);
        }
      },
      merge: (persistedState, currentState) => {
        const state = persistedState as Partial<AuthStore> | undefined;
        const cookieToken = getCookieToken();
        const cookieCustomer = getCookieCustomer();
        return {
          ...currentState,
          ...state,
          token: cookieToken || state?.token || null,
          customer: cookieCustomer || state?.customer || null,
          isAuthenticated: !!(cookieToken || cookieCustomer || state?.token || state?.customer),
          companyId: state?.companyId || COMPANY_ID,
        };
      },
    }
  )
);
```

**How it works — the dual persistence model**

- The auth state is written to **two** places: Zustand's `persist` middleware (`localStorage["auth-storage"]`) **and** cookies.
- **Cookies win.** `merge()` prefers the cookie values over the persisted ones, and `onRehydrateStorage` re-applies them after localStorage is read back.
- Why? Cookies survive a `localStorage` clear and can be read server-side later.
- `setToken` / `setCustomer` always keep the cookie and the store in sync — no way to have one without the other.
- `logout()` deletes both cookies and nulls the state.
- `COMPANY_ID` comes from `VITE_COMPANY_ID` and is attached to every auth + CRM request.

## `src/features/auth/services/authAPI.ts` — every auth endpoint (260 lines)

```
const API_URL = import.meta.env.VITE_API_URL || "{{url}}";

export const authAPI = {
  login: async (payload: LoginPayload): Promise<LoginResponse> => {
    const response = await axios.post(`${API_URL}/customers/login`, payload);
    return response.data;
  },

  signUp: async (payload: SignUpPayload): Promise<SignUpResponse> => {
    const response = await axios.post(`${API_URL}/customers/register`, payload);
    return response.data;
  },

  verifyEmail: async (payload: VerifyEmailPayload): Promise<VerifyEmailResponse> => {
    const response = await axios.post(`${API_URL}/customers/verify-email`, payload);
    return response.data;
  },

  resendOTP: async (payload: ResendOTPPayload): Promise<ResendOTPResponse> => {
    const response = await axios.post(`${API_URL}/customers/resend-verification-otp`, payload);
```

```
    return response.data;
  },

  forgotPassword: async (payload: ForgotPasswordPayload): Promise<ForgotPasswordResponse> => {
    const response = await axios.post(`${API_URL}/customers/forgot-password`, payload);
    return response.data;
  },

  resetPassword: async (payload: ResetPasswordPayload): Promise<ResetPasswordResponse> => {
    const response = await axios.post(`${API_URL}/customers/reset-password`, payload);
    return response.data;
  },

  getProfile: async (): Promise<ProfileResponse> => {
    const token = useAuthStore.getState().token;         // read the token imperatively
    const response = await axios.get(`${API_URL}/customers/profile`, {
      headers: { Authorization: `Bearer ${token}` },
    });
    return response.data;
  },

  updateProfile: async (payload: UpdateProfilePayload): Promise<UpdateProfileResponse> => {
    const token = useAuthStore.getState().token;
    const response = await axios.put(`${API_URL}/customers/profile`, payload, {
      headers: { Authorization: `Bearer ${token}` },
    });
    return response.data;
  },

  changePassword: async (payload: ChangePasswordPayload): Promise<ChangePasswordResponse> => {
    const token = useAuthStore.getState().token;
    const response = await axios.put(`${API_URL}/customers/change-password`, payload, {
      headers: { Authorization: `Bearer ${token}` },
    });
    return response.data;
  },
};
```

### Endpoint map

| Method | Endpoint | Auth |
|---|---|---|
| POST | `/customers/login` | — |
| POST | `/customers/register` | — |
| POST | `/customers/verify-email` | — |
| POST | `/customers/resend-verification-otp` | — |
| POST | `/customers/forgot-password` | — |
| POST | `/customers/reset-password` | — |
| GET | `/customers/profile` | Bearer |
| PUT | `/customers/profile` | Bearer |
| PUT | `/customers/change-password` | Bearer |

### How it works

- There is **no axios interceptor**. Each method pulls the token itself with `useAuthStore.getState().token` and sets the `Authorization` header by hand.
- Every payload includes `companyId` (multi-tenant).
- `customFieldsAPI`, the CRM posts and the sales calls all use **plain `axios`**, not this service.

## `src/features/auth/pages/Login.tsx` — login *and* sign-up in one page

One component, two modes toggled by `isSignUp`.

```
const isSignInValid = email && password;

const isSignUpValid =
  firstName && lastName && phoneNumber && email && password && confirmPassword &&
  password === confirmPassword && agree;

const handleLogin = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!isSignInValid) return;

  setIsLoading(true);
  try {
    const response = await authAPI.login({ companyId, email, password });

    if (response.token) {
      setToken(response.token);                // -> cookie + store
      setCustomer(response.customer);          // -> cookie + store
      setCompanyId(response.customer.companyId);
      await usePropertyStore.getState().fetchWishlist();   // fill the shortlist badge
      toast.success("Login successful!");
      navigate("/");
    }
  } catch (error: unknown) {
    toast.error(getErrorMessage(error, "Login failed. Please try again."));
    console.error("Login error:", error);
  } finally {
    setIsLoading(false);
  }
};
const handleSignUp = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!isSignUpValid) return;

  setIsLoading(true);
  try {
    const response = await authAPI.signUp({
      companyId, firstName, lastName, phoneNumber, email, password,
    });

    if (response) {
      if (response.companyId) setCompanyId(response.companyId);
      toast.success("OTP sent to your email!");
      sessionStorage.setItem(
        "pendingVerification",
        JSON.stringify({ email, firstName, lastName })
      );
      navigate("/auth/verify-email");
    }
  } catch (error: unknown) {
    toast.error(getErrorMessage(error, "Sign up failed. Please try again."));
  } finally {
    setIsLoading(false);
  }
};
```

### How it works

- The sign-up flow does **not** log you in — it stashes the email in `sessionStorage["pendingVerification"]` and sends you to the OTP page.
- `getErrorMessage` is a local helper repeated in all 5 auth pages (it unwraps `error.response.data.message`).

## `src/features/auth/pages/VerifyEmail.tsx` — 6-box OTP (308 lines)

```
const [otp, setOtp] = useState(["", "", "", "", "", ""]);
const [timeLeft, setTimeLeft] = useState(900);   // 15 minutes
const [canResend, setCanResend] = useState(false);

useEffect(() => {
  const data = sessionStorage.getItem("pendingVerification");
  if (!data) { navigate("/auth/login"); return; }   // no pending signup -> bail
  setPendingData(JSON.parse(data));
}, [navigate]);

// 1-second countdown
useEffect(() => {
  if (timeLeft <= 0) { setCanResend(true); return; }
  const timer = setInterval(() => setTimeLeft((prev) => prev - 1), 1000);
  return () => clearInterval(timer);
}, [timeLeft]);

const handleOtpChange = (index: number, value: string) => {
  if (value.length > 1) return;                    // one digit per box
  const newOtp = [...otp];
  newOtp[index] = value;
  setOtp(newOtp);
  if (value && index < 5) {                        // auto-advance
    document.getElementById(`otp-${index + 1}`)?.focus();
  }
};

const handleKeyDown = (index: number, e: React.KeyboardEvent<HTMLInputElement>) => {
  if (e.key === "Backspace" && !otp[index] && index > 0) {   // auto-retreat
    document.getElementById(`otp-${index - 1}`)?.focus();
  }
};

const handleVerifyEmail = async (e: React.FormEvent) => {
  e.preventDefault();
  const otpCode = otp.join("");
  if (otpCode.length !== 6) { toast.error("Please enter all 6 digits"); return; }

  const response = await authAPI.verifyEmail({ companyId, email: pendingData.email, otp: otpCode });

  if (response.token && response.customer) {
    setToken(response.token);
    setCustomer(response.customer);
    setCompanyId(response.customer.companyId);
    toast.success("Email verified successfully!");
    sessionStorage.removeItem("pendingVerification");
    navigate("/");
  }
};
```

### How it works

- The 6 boxes are independent `<input>`s; the component moves focus between them by DOM id (`otp-0` … `otp-5`) — no refs needed.
- The resend button unlocks after the countdown, and resets the timer to 5 minutes.

## `src/features/auth/pages/ForgotPassword.tsx` — two-step wizard (326 lines)

```
const [step, setStep] = useState<"email" | "otp">("email");

const sendResetCode = useCallback(async () => {
  if (!isEmailValid || hasSentCode || isLoading) return;
  setIsLoading(true);
  try {
    await authAPI.forgotPassword({ companyId, email });
    toast.success("Reset code sent to your email!");
    sessionStorage.setItem("resetEmail", email);
    setHasSentCode(true);
    setStep("otp");
  } catch (error) {
    /* toast + console.error */
  } finally { setIsLoading(false); }
}, [companyId, email, hasSentCode, isEmailValid, isLoading]);

const handleVerifyOtp = () => {
  if (!isOtpComplete) { setFormError("Please enter all 6 digits of the code."); return; }
  sessionStorage.setItem("resetEmail", email);
  sessionStorage.setItem("resetOtp", otp.join(""));
  navigate("/auth/reset-password");
};
```

### How it works

- Step 1 sends the code, step 2 collects the OTP.
- The OTP is **not verified against the server here** — it is just handed to the next page via `sessionStorage` and verified as part of `reset-password`.

## `src/features/auth/pages/ResetPassword.tsx` (213 lines)

```
useEffect(() => {
  const resetEmail = sessionStorage.getItem("resetEmail");
  const otpFromStorage = sessionStorage.getItem("resetOtp");
  if (!resetEmail || !otpFromStorage) {
    navigate("/auth/forgot-password");   // arrived without the wizard
    return;
  }
  setEmail(resetEmail);
  setResetOtp(otpFromStorage);
}, [navigate]);

const handleResetPassword = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!isPasswordValid || !email || !resetOtp) { setFormError("Please complete the password fields."); return; }
  if (newPassword !== confirmPassword) { setFormError("Passwords do not match"); return; }
  if (newPassword.length < 8) { setFormError("Password must be at least 8 characters"); return; }

  await authAPI.resetPassword({ companyId, email, otp: resetOtp, newPassword });
  toast.success("Password reset successfully!");
  sessionStorage.removeItem("resetEmail");
  navigate("/auth/login");
};
```

## `src/features/auth/pages/Profile.tsx` — two tabs (747 lines)

```
const [activeTab, setActiveTab] = useState<"info" | "security">("info");
const [editingSection, setEditingSection] = useState<Section | null>(null);  // "personal" | "address"

// Load once on mount
useEffect(() => {
  const fetchProfile = async () => {
    setIsLoading(true);
    try {
      const data = await authAPI.getProfile();
      loadFromProfile(data);
    } catch (error) {
      console.error("Failed to load profile:", error);
      toast.error("Failed to load profile details.");
    } finally { setIsLoading(false); }
  };
  fetchProfile();
}, []);

// Save a section (Personal or Address)
const handleSaveSection = async (section: Section) => {
  if (section === "personal" && (!firstName || !lastName)) {
    toast.error("First and Last name are required."); return;
  }
  setIsSaving(true);
  try {
    const payload = {
      firstName, lastName, phoneNumber,
      dateOfBirth: formatDateForPayload(dateOfBirth),
      address, city, state, country, postalCode, profilePicture,
    };
    const response = await authAPI.updateProfile(payload);
    toast.success(response.message || "Profile updated successfully!");

    if (response.customer) {
      loadFromProfile(response.customer);
      setCustomer({ /* mirror the server shape into the auth store */ });
      setCompanyId(response.customer.companyId);
    }
    setEditingSection(null);
  } catch (error) {
    toast.error(getErrorMessage(error, "Failed to update profile."));
  } finally { setIsSaving(false); }
};

// Photo upload — FileReader -> data URL -> PUT /customers/profile
const handlePhotoSelect = async (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  e.target.value = "";
  if (!file) return;
  if (!file.type.startsWith("image/")) { toast.error("Please select an image file."); return; }
  if (file.size > 5 * 1024 * 1024) { toast.error("Image must be smaller than 5MB."); return; }

  const reader = new FileReader();
  reader.onload = async () => {
    const dataUrl = reader.result as string;
    setProfilePicture(dataUrl);                 // instant local preview
    setIsUploadingPhoto(true);
    try {
      const response = await authAPI.updateProfile({
        firstName, lastName, phoneNumber,
        dateOfBirth: formatDateForPayload(dateOfBirth),
        address, city, state, country, postalCode,
        profilePicture: dataUrl,                // base64 in the JSON body
      });
      toast.success("Profile photo updated!");
      if (response.customer) loadFromProfile(response.customer);
```

```
    } catch (error) {
      toast.error(getErrorMessage(error, "Failed to upload photo."));
    } finally { setIsUploadingPhoto(false); }
  };
  reader.onerror = () => toast.error("Couldn't read that image.");
  reader.readAsDataURL(file);
};
```

**Date helpers** (the API uses `DD-MM-YYYY`, `<input type="date">` uses `YYYY-MM-DD`):

```
const formatDateForInput = (dateStr?: string) => { /* -> YYYY-MM-DD */ };
const formatDateForPayload = (dateStr: string) => { /* -> DD-MM-YYYY */ };
const formatDateForDisplay = (dateStr?: string) => { /* -> DD-MM-YYYY */ };
```

**Password rules** enforced in `handleChangePassword`: all fields required, `newPassword.length >= 8`, new must equal confirm, new must differ from current.

**How it works**

- Only one section can be in edit mode at a time (`editingSection`).
- `cancelEdit()` re-loads the last server snapshot, discarding local edits.
- The photo is stored as a **base64 data URL inside the JSON body** — there is no file-upload endpoint.

## `src/features/auth/components/ProfileDropdown.tsx` (76 lines)

```
function ProfileDropdown({ isOpen, onClose }: ProfileDropdownProps) {
  const { customer, logout } = useAuthStore();
  const navigate = useNavigate();

  const initials =
    `${firstName.charAt(0)}${lastName.charAt(0)}`.toUpperCase() || "U";

  const handleLogout = () => {
    logout();
    onClose();
    navigate("/");
  };
  // ... avatar + name/email header, "View Profile" link, "Logout" button
}
```

The dropdown is always mounted; `isOpen` only toggles opacity/scale classes so the open *and* close transitions animate.

# 5. Properties: the store (the core of the app)

`src/features/properties/store/usePropertyStore.ts` — 705 lines. This is where almost every interesting thing happens.

## 5.1 The API client and small helpers

```
const sabiFlowApi = axios.create({
  baseURL: "https://api.sabiflow.com/api/inventory/portal/rewacity",
});

const emptyPricing = {
  LegalFee: 0, ServiceFee: 0, CautionFee: 0,
  PropertyCost: 0, AgentFee: 0, TotalCost: 0,
};

const emptyGeoLocation = { lat: 0, lng: 0 };

/** Returns the first truthy value, or "" if none are truthy. */
const firstTruthy = (...values: (string | undefined | null)[]): string =>
  values.find((v) => !!v) ?? "";
/** Merge string arrays, trim, de-duplicate case-insensitively. */
const collectStringItems = (...groups: (string[] | undefined)[]): string[] => {
  const seen = new Set<string>();
  return groups.flatMap((group) =>
    (group ?? []).flatMap((item) => {
      const value = item.trim();
      const normalized = value.toLowerCase();
      if (!value || seen.has(normalized)) return [];
      seen.add(normalized);
      return [value];
    }),
  );
};

/** Safe localStorage JSON read/write (never throws). */
const readJSON = <T,>(key: string, fallback: T): T => {
  if (typeof window === "undefined") return fallback;
  try {
    const raw = window.localStorage.getItem(key);
    return raw ? (JSON.parse(raw) as T) : fallback;
  } catch (error) {
    console.error(`Failed to read "${key}" from localStorage:`, error);
    return fallback;
  }
};

const writeJSON = (key: string, value: unknown) => {
  if (typeof window === "undefined") return;
  try {
    window.localStorage.setItem(key, JSON.stringify(value));
  } catch (error) {
    console.error(`Failed to write "${key}" to localStorage:`, error);
  }
};
```

**Note:** the inventory base URL is **hard-coded** here (not `VITE_API_URL`).

## 5.2 The mapper — API shape → app shape

This is the single most important function in the codebase.

```
const mapSabiFlowProductsToProperties = (items: SabiFlowProduct[]): Property[] =>
  items.map((item) => {
    const customData = item.customData;
    const apiPricing = customData?.pricing;

    const propertyCost = apiPricing?.property_cost || 0;
    const agentFee     = apiPricing?.agent_fee || 0;
    const legalFee     = apiPricing?.legal_fee || 0;
    const cautionFee   = apiPricing?.caution_fee || 0;

    const creatorClassification = firstTruthy(
      item.creatorClassification,
      typeof item.createdBy === "object" && item.createdBy !== null
        ? item.createdBy.classification ?? undefined
        : undefined,
      item.customData?.creatorClassification
    );

    // Agents get an extra 5% service fee on top of whatever the API returned.
    const isAgent = creatorClassification.toLowerCase() === "agent";
    const serviceFee = (apiPricing?.service_fee || 0) + (isAgent ? propertyCost * 0.05 : 0);

    const totalPrice = propertyCost + agentFee + legalFee + serviceFee + cautionFee;

    // --- rules / features / bills: the API sends EITHER an array OR an object ---
    const rawRules      = customData?.rules;
    const rawFeatures   = customData?.key_features_and_amenities;
    const featuresAndAmenities = customData?.features_and_amenities;
    const billsAndCharges      = customData?.bills_and_charges;

    const keyFeatures = collectStringItems(
      Array.isArray(rawFeatures) ? rawFeatures : undefined,
      featuresAndAmenities?.key_features_and_amenities,
    );
    const additionalFeatures = collectStringItems(
      featuresAndAmenities?.additional_features_and_amenities,
      !Array.isArray(rawFeatures) ? rawFeatures?.other_features_and_amenities : undefined,
    );
    const rules = Array.isArray(rawRules)
      ? collectStringItems(rawRules)
      : collectStringItems(rawRules?.key_rules);
    const additionalRules = Array.isArray(rawRules)
      ? []
      : collectStringItems(rawRules?.additional_rules);
    const primaryBillsAndCharges = collectStringItems(
      billsAndCharges?.baic_bills_and_charges,       // note: API typo "baic"
      billsAndCharges?.basic_bills_and_charges,     // correct spelling, checked too
    );
    const additionalBillsAndCharges = collectStringItems(
      billsAndCharges?.additional_bills_and_charges,
    );

    // createdBy can be an object OR a plain string id
    const normalizedCreatedBy =
      item.createdBy && typeof item.createdBy === "object"
        ? {
            _id: item.createdBy._id ?? "",
            id: item.createdBy.id ?? item.createdBy._id ?? "",
            firstName: item.createdBy.firstName ?? "",
            lastName: item.createdBy.lastName ?? "",
            email: item.createdBy.email ?? undefined,
            classification: item.createdBy.classification ?? null,
          }
        : null;
```

```
  return {
    id: item._id,
    sku: item.sku,
    slug: item.slug,
    name: item.name,
    img: ensureHttps(item.thumbnail || item.images[0] || ""),
    images: (item.images || []).map(ensureHttps),
    description: item.description || "",
    bedrooms: customData?.bedrooms || 0,
    bathrooms: customData?.bathrooms || 0,
    category: item.categoryId?.name || "Property",
    duration: customData?.duration || "",
    listingType: customData?.listing_type || "",
    rules,
    additionalRules,
    pricing: {
      PropertyCost: propertyCost,
      AgentFee: agentFee,
      LegalFee: legalFee,
      ServiceFee: serviceFee,
      CautionFee: cautionFee,
      TotalCost: totalPrice || item.price || 0,
    },
    createdBy: normalizedCreatedBy,
    creatorClassification: creatorClassification || null,
    location: {
      area: customData?.location?.area || "",
      city: customData?.location?.city || "",
      city_town: customData?.location?.city_town || "",
      state: customData?.location?.state || "",
      nearest_university: customData?.location?.nearest_university || "",
    },
    geo_location: {
      lat: customData?.geo_location?.lat || 0,
      lng: customData?.geo_location?.lng || 0,
      address: customData?.geo_location?.address || "",
    },
    yearBuilt: customData?.yearBuilt || 0,
    keyFeatures,
    additionalFeatures,
    billsAndCharges: primaryBillsAndCharges,
    additionalBillsAndCharges,
    specialNotes: customData?.special_notes || [],
    specifications: [
      ...(item.specifications
        ? Object.entries(item.specifications).map(([label, value]) => ({ label, value }))
        : []),
      ...(customData?.specifications || []),
    ],
    videoUrl: ensureHttps(item.videoUrl || ""),
    caretakerContact: customData?.care_taker_contact_optional
      ? {
          whatsapp: customData.care_taker_contact_optional.wattsapp_contact, // API typo
          phone: customData.care_taker_contact_optional.call_contact,
        }
      : undefined,
    visitationfee: (customData?.visitation_fee || 0) + 500,   // +₦500 platform charge
  };
});
```

**How it works**

- Every field is defensive: `|| ""`, `|| 0`, `?? []` — the API omits fields often.
- `TotalCost` is recomputed client-side: `property + agent + legal + service + caution`. It only falls back to `item.price` if the computed total is 0.
- **The 5% agent surcharge** is a business rule that lives in the client, not the API.

- `rules`, `key_features_and_amenities` and `bills_and_charges` are accepted in **two shapes** (flat array *or* object of arrays) — that is why `collectStringItems` and the `Array.isArray` checks exist.
- `ensureHttps` is applied to every image URL and the video URL.
- `visitationfee` always includes the extra ₦500 RewaCiti service charge.

### 5.3 Wishlist metadata cache

The wishlist endpoint sometimes returns products without `name` / `slug` / `img` / `location`. The store caches them locally so the shortlist always renders nicely.

```
const WISHLIST_METADATA_STORAGE_KEY = "rewaciti_wishlist_metadata";

/** Fills in any missing name/slug/img/location fields from the cache. */
const mergeWishlistMetadata = (property: Property, storedMetadata: WishlistMetadataMap): Property => {
  const saved = storedMetadata[property.id];
  if (!saved) return property;

  return {
    ...property,
    name: firstTruthy(property.name, saved.name),
    slug: firstTruthy(property.slug, saved.slug),
    img: firstTruthy(property.img, saved.img),
    location: {
      area: firstTruthy(property.location.area, saved.location?.area),
      city: firstTruthy(property.location.city, saved.location?.city),
      city_town: firstTruthy(property.location.city_town, saved.location?.city_town),
      state: firstTruthy(property.location.state, saved.location?.state),
      nearest_university: firstTruthy(property.location.nearest_university, saved.location?.nearest_university),
    },
  };
};

const syncWishlistMetadata = (properties: Property[]) => {
  const storedMetadata = readStoredWishlistMetadata();
  const nextMetadata = properties.reduce<WishlistMetadataMap>((acc, property) => {
    acc[property.id] = buildWishlistMetadataEntry(property, storedMetadata[property.id]);
    return acc;
  }, {});
  writeStoredWishlistMetadata(nextMetadata);
};
```

### 5.4 Fetching the list

```
fetchProperties: async (apiPage = 1, filters?: Record<string, string | number | undefined>) => {
  set({ loading: true, error: null });

  try {
    const resolvedFilters = filters ?? get().filters;

    // Drop empty filters so we don't send ?state=&city= etc.
    const params = Object.entries(resolvedFilters).reduce<Record<string, string>>(
      (acc, [key, value]) => {
        if (value !== undefined && value !== null && value !== "") {
          acc[key] = String(value);
        }
        return acc;
      },
      {}
    );

    const res = await sabiFlowApi.get<{ data: SabiFlowProduct[]; total?: number }>("/products", {
```

```
      params: { page: apiPage, limit: 30, ...params },
    });

    const properties = mapSabiFlowProductsToProperties(res.data.data);

    set({
      properties,
      filteredProperties: properties,
      loading: false,
      apiPage,
      page: apiPage - 1,               // UI is 0-based
      totalProperties: res.data.total || properties.length,
      filters: resolvedFilters,
    });
  } catch (err) {
    console.error(err);
    set({ error: "Failed to load property data", loading: false });
  }
},
```

**How it works**

- Pagination is **server-side**: `?page=N&limit=30`.
- `filters` are passed straight through as query params — the API understands `customData.location.state`, `minPrice`, `maxPrice`, `categoryId`, etc.
- Empty strings are stripped so the API doesn't treat `""` as a filter value.

## 5.5 Other read actions

```
fetchFeaturedProperties: async () => {
  // Skip if already loaded — avoid re-fetching on every home page visit
  if (get().featuredProperties.length > 0) return;
  set({ featuredLoading: true });
  try {
    const res = await sabiFlowApi.get<{ data: SabiFlowProduct[]; total?: number }>("/products", {
      params: { page: 1, limit: 8 },
    });
    set({ featuredProperties: mapSabiFlowProductsToProperties(res.data.data), featuredLoading: false });
  } catch (err) { /* console.error + featuredLoading: false */ }
},

fetchPropertyFees: async () => {
  try {
    const res = await axios.get<PropertyPaymentFees>("/data/PropertyPaymentFees.json");
    set({ fees: res.data });
  } catch (err) {
    console.error("Failed to fetch property payment fees", err);
  }
},

fetchCategories: async () => {
  set({ categoriesLoading: true });
  try {
    const res = await sabiFlowApi.get<Category[]>("/categories");
    set({ categories: res.data, categoriesLoading: false });
  } catch (error) { /* ... */ }
},

fetchFilters: async () => {
  set({ filtersLoading: true });
  try {
    const res = await sabiFlowApi.get("/filters");
    set({ filtersData: res.data, filtersLoading: false });
```

```
  } catch (err) { /* ... */ }
},
```

`fetchPropertyFees()` requests `/data/PropertyPaymentFees.json` — **that file does not exist** in `public/data/`, so the request always 404s and is swallowed.

## 5.6 Related properties

```
fetchRelatedProperties: async (property: Property, sameAgentOnly = false, page = 1, limit = 30) => {
  set({ relatedPropertiesLoading: true });

  try {
    let { categories } = get();
    if (!categories.length) {
      await get().fetchCategories();       // lazily load categories
      categories = get().categories;
    }

    // The property only has a category NAME, but the API wants an id
    const categoryId = categories.find(
      (c) => c.name.trim().toLowerCase() === property.category.trim().toLowerCase()
    )?.id;

    const agentId = property.createdBy?._id || property.createdBy?.id;

    const params: Record<string, string | number> = { page, limit };
    if (categoryId) params.categoryId = categoryId;
    if (property.location.state) params["customData.location.state"] = property.location.state;
    if (property.location.city_town) params["customData.location.city_town"] = property.location.city_town;
    if (sameAgentOnly && agentId) params.createdBy = agentId;

    const res = await sabiFlowApi.get<{ data: SabiFlowProduct[]; total?: number }>("/products", { params });

    const mappedProperties = mapSabiFlowProductsToProperties(res.data.data);
    const relatedProperties = mappedProperties.filter((p) => p.id !== property.id);  // drop itself

    set({ relatedProperties, totalRelatedProperties: res.data.total ?? mappedProperties.length,
          relatedPropertiesLoading: false });
  } catch (err) { /* reset to empty */ }
},
```

### How it works

- ”Same agent” is just an extra `createdBy=<id>` query param.
- The current property is filtered out client-side.

## 5.7 Detail page: cache-first loading

```
fetchPropertyBySlug: async (slug: string) => {
  // Serve from cache first so back-navigation is instant,
  // but still revalidate in the background for fresh data.
  const { properties, relatedProperties, featuredProperties, currentProperty } = get();
  const cached =
    (currentProperty?.slug === slug ? currentProperty : null) ??
    properties.find((p) => p.slug === slug) ??
    relatedProperties.find((p) => p.slug === slug) ??
    featuredProperties.find((p) => p.slug === slug) ??
    null;

  if (cached) {
    set({ currentProperty: cached });          // render immediately
```

```
  } else {
    set({ detailsLoading: true, currentProperty: null });
  }

  try {
    const res = await sabiFlowApi.get<SabiFlowProduct>(`/products/${slug}`);
    const raw = res.data as SabiFlowProduct | { data?: SabiFlowProduct };
    const item = (raw as { data?: SabiFlowProduct }).data ?? (raw as SabiFlowProduct);
    if (!item || !item._id) throw new Error("Property not found");

    const [mapped] = mapSabiFlowProductsToProperties([item]);
    set((state) => ({
      currentProperty: mapped,
      detailsLoading: false,
      // Upsert into the list cache so find() also works on revisit.
      properties: state.properties.some((p) => p.slug === mapped.slug)
        ? state.properties.map((p) => (p.slug === mapped.slug ? mapped : p))
        : [...state.properties, mapped],
    }));
  } catch (err) {
    console.error(`Failed to fetch property "${slug}":`, err);
    // If we had a cached copy, keep it; otherwise clear so UI shows Not Found.
    set({ detailsLoading: false, ...(cached ? {} : { currentProperty: null }) });
  }
},
```

**This is a stale-while-revalidate pattern**: show the cached property immediately, then silently replace it with fresh server data when the request returns.

## 5.8 Pagination

```
nextPage: () => {
  const { apiPage, totalProperties, ITEMS_PER_PAGE, fetchProperties, filters } = get();
  const maxPage = Math.max(1, Math.ceil(totalProperties / ITEMS_PER_PAGE));
  const nextPageNumber = Math.min(apiPage + 1, maxPage);

  if (nextPageNumber > apiPage) {
    set({ page: nextPageNumber - 1, apiPage: nextPageNumber });
    void fetchProperties(nextPageNumber, filters);
  }
},

prevPage: () => {
  const { apiPage, fetchProperties, filters } = get();
  const prevPageNumber = Math.max(apiPage - 1, 1);
  if (prevPageNumber < apiPage) {
    set({ page: prevPageNumber - 1, apiPage: prevPageNumber });
    void fetchProperties(prevPageNumber, filters);
  }
},

setPage: (page: number) => set({ page, apiPage: page + 1 }),
```

## 5.9 Client-side search

```
searchQuery: "",
setSearchQuery: (query: string) => {
  set({ searchQuery: query });
  const { properties } = get();
  if (!query) {
    set({ filteredProperties: properties });
  } else {
    const filtered = properties.filter((p) => propertyMatchesQuery(p, query));
    set({ filteredProperties: filtered, page: 0 });
  }
},
```

```
/** Case-insensitive substring match across a property's searchable text fields. */
const propertyMatchesQuery = (property: Property, query: string): boolean => {
  if (!query) return true;
  const q = query.toLowerCase();
  return (
    property.name.toLowerCase().includes(q) ||
    property.description.toLowerCase().includes(q) ||
    property.location.area.toLowerCase().includes(q) ||
    !!property.location.city_town?.toLowerCase().includes(q) ||
    property.location.state.toLowerCase().includes(q)
  );
};
```

This filters only the **current page** (30 items). The listing page's real search box sends a `search` query param to the API instead.

## 5.10 Wishlist: fetch + optimistic toggle

```
fetchWishlist: async () => {
  const { token, customer, isAuthenticated } = useAuthStore.getState();
  if (!isAuthenticated || !customer || !token) {
    set({ shortlistedProperties: [] });
    return;
  }

  const apiUrl = import.meta.env.VITE_API_URL || "{{url}}";

  try {
    const response = await axios.get<WishlistResponse>(`${apiUrl}/customers/wishlist`, {
      headers: { Authorization: `Bearer ${token}` },
    });

    const products = response.data?.products ?? [];
    const storedMetadata = readStoredWishlistMetadata();
    const mappedWishlist = products
      .map(mapWishlistProductToProperty)
      .map((property) => mergeWishlistMetadata(property, storedMetadata));

    const wishlistIds = mappedWishlist.map((property) => property.id);
    let availableInventoryIds = new Set<string>(wishlistIds);
    let shouldPreserveAll = false;

    try {
      availableInventoryIds = await fetchAvailableWishlistIds(wishlistIds);
    } catch (inventoryError) {
      console.warn("Wishlist availability lookup failed, keeping saved items for now:", inventoryError);
      shouldPreserveAll = true;
    }
```

```
    const nextShortlisted = shouldPreserveAll
      ? mappedWishlist
      : mappedWishlist.filter((property) => availableInventoryIds.has(property.id));

    // Delete items that no longer exist in the inventory
    if (!shouldPreserveAll && nextShortlisted.length < mappedWishlist.length) {
      const unavailableItems = mappedWishlist.filter((p) => !availableInventoryIds.has(p.id));
      await Promise.allSettled(
        unavailableItems.map((item) =>
          axios.delete(`${apiUrl}/customers/wishlist/${item.id}`, {
            headers: { Authorization: `Bearer ${token}` },
          })
        )
      );
    }

    syncWishlistMetadata(nextShortlisted);
    set({ shortlistedProperties: nextShortlisted });
  } catch (error) {
    console.error("Failed to fetch wishlist:", error);
  }
},
```

The availability check pages through the inventory to confirm each saved product still exists:

```
const fetchAvailableWishlistIds = async (
  wishlistIds: string[],
  maxPages = 5,
  pageSize = 100
): Promise<Set<string>> => {
  const idsToFind = new Set(wishlistIds);
  const foundIds = new Set<string>();
  let page = 1;
  while (page <= maxPages && foundIds.size < idsToFind.size) {
    const res = await sabiFlowApi.get<{ data: SabiFlowProduct[]; total?: number }>("/products", {
      params: { page, limit: pageSize },
    });
    const data = res.data.data ?? [];
    data.forEach((item) => { if (idsToFind.has(item._id)) foundIds.add(item._id); });
    if (data.length < pageSize) break;      // last page
    page += 1;
  }
  return foundIds;
};
```

The toggle is **optimistic** — the UI updates instantly, and rolls back on failure:

```
toggleShortlist: async (property: Property) => {
  const { token, customer, isAuthenticated } = useAuthStore.getState();
  if (!isAuthenticated || !customer || !token) return;

  const apiUrl = import.meta.env.VITE_API_URL || "{{url}}";
  const previousShortlisted = get().shortlistedProperties;
  const isShortlisted = previousShortlisted.some((p) => p.id === property.id);

  // Persist metadata immediately so location/name survive between sessions.
  const storedMetadata = readStoredWishlistMetadata();
  const nextStoredMetadata = {
    ...storedMetadata,
    [property.id]: buildWishlistMetadataEntry(property, storedMetadata[property.id]),
  };
  writeStoredWishlistMetadata(nextStoredMetadata);

  const enrichedProperty = mergeWishlistMetadata(property, nextStoredMetadata);
  const optimisticProperties = isShortlisted
    ? previousShortlisted.filter((p) => p.id !== property.id)
```

```
    : [...previousShortlisted, enrichedProperty];

  syncWishlistMetadata(optimisticProperties);
  set({ shortlistedProperties: optimisticProperties });      // <- UI updates now

  try {
    if (isShortlisted) {
      await axios.delete(`${apiUrl}/customers/wishlist/${property.id}`, {
        headers: { Authorization: `Bearer ${token}` },
      });
    } else {
      await axios.post(`${apiUrl}/customers/wishlist`,
        { productId: property.id },
        { headers: { Authorization: `Bearer ${token}` } }
      );
    }
  } catch (error) {
    console.error("Wishlist update failed:", error);
    set({ shortlistedProperties: previousShortlisted });    // <- roll back
    toast.error("Couldn't update wishlist. Please try again.");
  }
},
```

**How it works**

1. Update the local list immediately (optimistic).
2. Write the metadata cache so the item still renders after a reload.
3. Call the API.
4. On error → restore the previous list and show a toast.

## 5.11 Legacy client-side filter

```
filterProperties: (filters: {
  location?: string; category?: string; priceRange?: string; rooms?: number;
}) => {
  const { properties, searchQuery } = get();

  const filtered = properties.filter((p) => {
    const [minPrice, maxPrice] = filters.priceRange
      ? filters.priceRange.split("-").map(Number) : [0, Infinity];
    const price = p.pricing.TotalCost;

    const locationMatch = !filters.location ||
      [p.location.area, p.location.city_town, p.location.state].some((v) => v === filters.location);

    return (
      locationMatch &&
      propertyMatchesQuery(p, searchQuery) &&
      (!filters.category || p.category === filters.category) &&
      (!filters.rooms || p.bedrooms === filters.rooms) &&
      (!filters.priceRange || (maxPrice ? price >= minPrice && price <= maxPrice : price >= minPrice))
    );
  });

  set({ filteredProperties: filtered, page: 0 });
},
```

This is the **old** client-side filter. The live listing page uses server-side filters instead (see §6). It is still exported on the store.

# 6. Properties: listing page

`src/features/properties/pages/PropertiesPage.tsx` — 1151 lines.

## 6.1 State

```
const [searchTerm, setSearchTerm] = useState("");

const [state, setState] = useState("");
const [city, setCity] = useState("");
const [area, setArea] = useState("");
const [university, setUniversity] = useState("");
const [category, setCategory] = useState("");
const [bedrooms, setBedrooms] = useState("");
const [listingType, setListingType] = useState("");
const [selectedPriceLabel, setSelectedPriceLabel] = useState("");

const [minPriceInput, setMinPriceInput] = useState("");
const [maxPriceInput, setMaxPriceInput] = useState("");
const [priceRange, setPriceRange] = useState<[number, number]>([0, 999999999]);

const [bedroomCount, setBedroomCount] = useState(0);
const [sharedRoomOnly, setSharedRoomOnly] = useState(false);

const [showFilters, setShowFilters] = useState(false);
const [isPreferenceModalOpen, setIsPreferenceModalOpen] = useState(false);
const [isCoRentModalOpen, setIsCoRentModalOpen] = useState(false);
const [isListPropertyOpen, setIsListPropertyOpen] = useState(false);
const [viewMode, setViewMode] = useState<"list" | "map">("list");
const [hoveredPropertyId, setHoveredPropertyId] = useState<string | null>(null);
const [listingTypes, setListingTypes] = useState<string[]>([]);
```

## 6.2 Measuring the sticky bar (for the map offset)

```
const NAVBAR_HEIGHT_PX = 64;
const topBarRef = useRef<HTMLDivElement>(null);
const [mapTopOffset, setMapTopOffset] = useState(NAVBAR_HEIGHT_PX + 64);

useLayoutEffect(() => {
  const measure = () => {
    if (topBarRef.current) {
      setMapTopOffset(NAVBAR_HEIGHT_PX + topBarRef.current.offsetHeight);
    }
  };
  measure();
  window.addEventListener("resize", measure);
  const raf = requestAnimationFrame(measure);      // re-measure after layout
  return () => {
    window.removeEventListener("resize", measure);
    cancelAnimationFrame(raf);
  };
}, []);
```

**How it works** — the map panel is `position: sticky` and must start exactly below the sticky filter bar. Rather than hard-coding a height, the bar's real height is measured and fed into the map's `top` / `height` inline styles.

### 6.3 iOS-safe body scroll lock

```
useEffect(() => {
  if (showFilters) {
    const scrollY = window.scrollY;
    document.body.style.position = "fixed";
    document.body.style.top = `-${scrollY}px`;   // remember where we were
    document.body.style.left = "0";
    document.body.style.right = "0";
    document.body.style.width = "100%";
  } else {
    const scrollY = document.body.style.top;
    document.body.style.position = "";
    document.body.style.top = "";
    document.body.style.left = "";
    document.body.style.right = "";
    document.body.style.width = "";
    if (scrollY) {
      window.scrollTo(0, parseInt(scrollY || "0", 10) * -1);   // restore
    }
  }
  return () => { /* always reset on unmount */ };
}, [showFilters]);
```

**Why not `overflow: hidden`?** On iOS Safari that does not stop the page scrolling behind the sheet. Fixing the body and restoring `scrollY` on close is the reliable way.

### 6.4 The one effect that fetches everything

```
useEffect(() => {
  const isCategoryId = categories.some(
    (option) => String(option.id) === String(category),
  );

  fetchProperties(1, {
    search: searchTerm || undefined,
    "customData.location.state": state || undefined,
    "customData.location.city_town": city || undefined,
    "customData.location.area": area || undefined,
    "customData.location.nearest_institution_in_full": university || undefined,
    ...(category ? (isCategoryId ? { categoryId: category } : { category }) : {}),
    "customData.bedrooms": bedrooms || undefined,
    minPrice: priceRange[0] > 0 ? priceRange[0] : undefined,
    maxPrice: priceRange[1] < 999999999 ? priceRange[1] : undefined,
    "customData.listing_type": listingType || undefined,
  });
}, [searchTerm, category, bedrooms, state, city, area, university,
    priceRange, fetchProperties, listingType, categories]);
```

**UI control → query param**

| UI control | Query param |
|---|---|
| Search box | `search` |
| State | `customData.location.state` |
| City | `customData.location.city_town` |
| Area | `customData.location.area` |
| University | `customData.location.nearest_institution_in_full` |
| Category | `categoryId` (if it matches a category id) else `category` |
| Bedrooms | `customData.bedrooms` |
| Min price | `minPrice` |
| Max price | `maxPrice` |

| UI control | Query param |
|---|---|
| Listing type | `customData.listing_type` |

**How it works**

- Any filter change resets to page 1 (`fetchProperties(1, …)`).
- `undefined` values are stripped by the store before the request is sent.

## 6.5 Two-way sync between the stepper and the bedroom filter

```
// stepper / toggle -> bedrooms string
useEffect(() => {
  if (sharedRoomOnly) setBedrooms("shared");
  else if (bedroomCount > 0) setBedrooms(String(bedroomCount));
  else setBedrooms("");
}, [bedroomCount, sharedRoomOnly]);

// bedrooms string -> stepper / toggle
useEffect(() => {
  if (bedrooms === "shared") { setSharedRoomOnly(true); setBedroomCount(0); }
  else if (["1", "2", "3"].includes(bedrooms)) { setBedroomCount(Number(bedrooms)); setSharedRoomOnly(false); }
  else if (bedrooms === "") { setBedroomCount(0); setSharedRoomOnly(false); }
}, [bedrooms]);
```

Two effects that look redundant are actually needed because the bedrooms value can be set from **two** places (the desktop dropdown and the mobile filter sheet).

## 6.6 Price text → numbers

```
useEffect(() => {
  const min = minPriceInput.trim() === "" ? 0 : Math.max(0, Number(minPriceInput));
  const max = maxPriceInput.trim() === "" ? 999999999 : Math.max(0, Number(maxPriceInput));
  if (Number.isNaN(min) || Number.isNaN(max)) return;
  setPriceRange([min, max]);
}, [minPriceInput, maxPriceInput]);
```

`999999999` is the sentinel for ”no max”. The fetch effect only sends `maxPrice` when it is smaller than that sentinel.

## 6.7 Filter option builders

Every dropdown list is built with `useMemo` from two sources: the **location hierarchy** (derived from real products) and the **custom-field values** (from the API). Both are merged case-insensitively and sorted.

```
const stateOptions = useMemo(() => {
  const stateMap = new Map<string, string>();     // lowercased -> original

  locationHierarchy.forEach((item) => {
    if (item.state) {
      const lower = item.state.toLowerCase();
      if (!stateMap.has(lower)) stateMap.set(lower, item.state);
    }
  });

  (customFilterValues?.["location.state"] || []).forEach((s) => {
    if (s) {
      const lower = s.toLowerCase();
```

```
      if (!stateMap.has(lower)) stateMap.set(lower, s);
    }
  });

  return Array.from(stateMap.entries())
    .sort((a, b) => a[1].localeCompare(b[1]))
    .map(([_, val]) => ({ label: val, value: val }));
}, [locationHierarchy, customFilterValues]);
```

`cityOptions`, `areaOptions` and `universityOptions` follow the same pattern but additionally filter by the currently selected state (and city for areas).

## 6.8 Cascading location logic

```
const handleStateChange = (selectedState: string) => {
  setState(selectedState);
  if (!selectedState) { setCity(""); setArea(""); return; }

  const stateLower = selectedState.toLowerCase();

  const validCities = locationHierarchy
    .filter((i) => i.state.toLowerCase() === stateLower)
    .map((i) => i.city.toLowerCase());
  if (city && !validCities.includes(city.toLowerCase())) setCity("");

  const validAreas = locationHierarchy
    .filter((i) => i.state.toLowerCase() === stateLower)
    .map((i) => i.area.toLowerCase());
  if (area && !validAreas.includes(area.toLowerCase())) setArea("");

  const validUnis = locationHierarchy
    .filter((i) => i.state.toLowerCase() === stateLower)
    .map((i) => (i.university || "").toLowerCase());
  if (university && !validUnis.includes(university.toLowerCase())) setUniversity("");
};

const handleCityChange = (selectedCity: string) => {
  setCity(selectedCity);
  if (!selectedCity) { setArea(""); return; }
  // clear the area if it does not exist in the new city
  const validAreas = locationHierarchy
    .filter((i) => i.state.toLowerCase() === state.toLowerCase() &&
                   i.city.toLowerCase() === selectedCity.toLowerCase())
    .map((i) => i.area.toLowerCase());
  if (area && !validAreas.includes(area.toLowerCase())) setArea("");
};

// Selecting an AREA reverse-derives the city
const handleAreaChange = (selectedArea: string) => {
  setArea(selectedArea);
  if (selectedArea && !city && state) {
    const match = locationHierarchy.find(
      (i) => i.state.toLowerCase() === state.toLowerCase() &&
             i.area.toLowerCase() === selectedArea.toLowerCase()
    );
    if (match?.city) setCity(match.city);
  }
};

// Selecting a UNIVERSITY reverse-derives both state and city
const handleUniversityChange = (selectedUniversity: string) => {
  setUniversity(selectedUniversity);
  if (selectedUniversity) {
    const match = locationHierarchy.find(
      (i) => (i.university || "").toLowerCase() === selectedUniversity.toLowerCase()
```

```
    );
    if (match) {
      if (match.state && match.state.toLowerCase() !== state.toLowerCase()) setState(match.state);
      if (match.city && match.city.toLowerCase() !== city.toLowerCase()) setCity(match.city);
    }
  }
};
```

**How it works** — the filters are a *tree*. Choosing a parent invalidates the children; choosing a leaf back-fills the parents. That keeps the query params consistent (you can never send `state=Osun` with `city=Lagos`).

### 6.9 Active filter badge + clear all

```
const activeFilterCount = [
  state, city, area, university, category,
  bedroomCount > 0 ? "x" : "",
  sharedRoomOnly ? "x" : "",
  selectedPriceLabel,
  minPriceInput.trim() !== "" ? "x" : "",
  maxPriceInput.trim() !== "" ? "x" : "",
  listingType,
].filter((v) => v !== "").length;

const clearAllFilters = () => {
  setState(""); setCity(""); setArea(""); setUniversity(""); setCategory("");
  setBedroomCount(0); setSharedRoomOnly(false);
  setMinPriceInput(""); setMaxPriceInput(""); setSelectedPriceLabel("");
  setListingType(""); setPriceRange([0, 999999999]); setBedrooms("");
};
```

### 6.10 List ↔ Map mode

```
{viewMode === "map" ? (
  loading ? (
    <div className="w-full h-[600px] rounded-xl ... animate-pulse">
      Loading interactive map...
    </div>
  ) : currentProperties.length === 0 ? (
    <div className="w-full h-[600px] rounded-xl ... text-center p-6">
      <h3>No properties found on the map</h3>
      <p>Try adjusting your search or filters to see results.</p>
    </div>
  ) : (
    <div className="flex flex-col lg:flex-row gap-4 items-start">
      {/* LEFT: the card list */}
      <div className="hidden lg:block lg:w-[42%] w-full">
        <div className="grid grid-cols-1 xl:grid-cols-2 gap-4">
          {currentProperties.map((property) => (
            <div
              key={property.id}
              id={`property-card-${property.id}`}
              onMouseEnter={() => setHoveredPropertyId(property.id)}
              onMouseLeave={() => setHoveredPropertyId(null)}
              className={`rounded-xl transition ${
                hoveredPropertyId === property.id
                  ? "ring-2 ring-[#703BF7]"
                  : "ring-1 ring-transparent"
              }`}
            >
              <PropertyCard property={property} infoLayout="stacked" />
            </div>
```

```
          ))}
        </div>
      </div>

      {/* RIGHT: the sticky map panel */}
      <div
        className="w-full lg:w-[58%] lg:sticky z-10"
        style={{
          top: `${mapTopOffset}px`,
          height: `calc(100vh - ${mapTopOffset}px)`,
          minHeight: 420,
        }}
      >
        <PropertyMap
          properties={currentProperties}
          heightClassName="h-full"
          hoveredPropertyId={hoveredPropertyId}
          onHoverProperty={setHoveredPropertyId}
          onSelectProperty={(id) => {
            setHoveredPropertyId(id);
            document.getElementById(`property-card-${id}`)?.scrollIntoView({
              behavior: "smooth", block: "center",
            });
          }}
        />
      </div>
    </div>
  )
) : (
  /* ...the normal list layout... */
)}
```

**How it works**

- `hoveredPropertyId` is a **two-way binding**: hovering a card highlights the marker, hovering a marker highlights the card.
- Clicking a marker smooth-scrolls the matching card into view by DOM id.
- Each card is wrapped in a `<div id="property-card-{id}">` purely so the map can find it.

## 6.11 The custom-fields service

`src/features/properties/services/customFieldsAPI.ts` (83 lines):

```ts
const API_URL = import.meta.env.VITE_API_URL || "{{url}}";
const COMPANY_SLUG = "rewacity";

export const customFieldsAPI = {
  getFilterValues: async (): Promise<CustomFieldsResponse> => {
    const keys = [
      "bedrooms", "bathrooms",
      "location.area", "location.city_town", "location.state",
      "location.nearest_institution_in_full",
      "listing_type",
    ].join(",");

    const response = await axios.get<CustomFieldsResponse>(
      `${API_URL}/inventory/portal/${COMPANY_SLUG}/custom-fields/values`,
      { params: { keys } }
    );
    return response.data;
  },

  getLocationHierarchy: async (): Promise<LocationHierarchyItem[]> => {
```

```
    try {
      const response = await axios.get<{ data?: Array<{ customData?: { location?: {...} } } }> }>(
        `${API_URL}/inventory/portal/${COMPANY_SLUG}/products`,
        { params: { limit: 200 } }
      );

      const list: LocationHierarchyItem[] = [];
      (response.data?.data || []).forEach((item) => {
        const loc = item.customData?.location;
        if (loc) {
          const state = (loc.state || "").trim();
          const city = (loc.city_town || loc.city || "").trim();
          const area = (loc.area || "").trim();
          const university = (loc.nearest_institution_in_full || "").trim();
          if (state || city || area || university) {
            list.push({ state, city, area, university });
          }
        }
      });
      return list;
    } catch (error) {
      console.error("Failed to fetch location hierarchy from products:", error);
      return [];
    }
  },
};
```

**How it works** — there is no dedicated ”locations” endpoint, so the hierarchy is **derived from the first 200 real products**. The `custom-fields/values` endpoint supplies the canonical list of states/cities/areas/universities.

# 7. Properties: detail page

`src/features/properties/pages/PropertyDetailsPage.tsx` — 1437 lines. It is one long scrolling document (not tabs).

## 7.1 Getting the property (cache-first, three sources)

```
const { slug } = useParams<{ slug: string }>();

const {
  properties, featuredProperties, loading, toggleShortlist, shortlistedProperties,
  relatedProperties, relatedPropertiesLoading, totalRelatedProperties,
  fetchRelatedProperties, fetchCategories, currentProperty, detailsLoading,
  fetchPropertyBySlug,
} = usePropertyStore();

// Look in every cache the store has
const cachedProperty =
  properties.find((p) => p.slug === slug) ??
  relatedProperties.find((p) => p.slug === slug) ??
  featuredProperties.find((p) => p.slug === slug) ??
  null;

const property =
  currentProperty && currentProperty.slug === slug
    ? currentProperty
    : cachedProperty ?? currentProperty;

const showDetailsSkeleton = detailsLoading || (!property && loading);

useEffect(() => {
  if (slug) void fetchPropertyBySlug(slug);
}, [slug, fetchPropertyBySlug]);

useEffect(() => { fetchCategories(); }, [fetchCategories]);

useEffect(() => { window.scrollTo(0, 0); }, [slug]);   // new property -> top
```

## 7.2 State

```
const [orientations, setOrientations] = useState<Record<number, "landscape" | "portrait">>({});
const [showActionBar, setShowActionBar] = useState(false);
const [showAllSpecifications, setShowAllSpecifications] = useState(false);
const [showAllSpecialNotes, setShowAllSpecialNotes] = useState(false);
const [activeFeatureModal, setActiveFeatureModal] = useState<"amenities" | "rules" | "bills" | null>(null);

const [currentIndex, setCurrentIndex] = useState(0);
const [step, setStep] = useState(
  typeof window !== "undefined" && window.innerWidth >= 640 ? 2 : 1,   // 2 images per row on desktop
);
const [isLightboxOpen, setIsLightboxOpen] = useState(false);
const [lightboxIndex, setLightboxIndex] = useState(0);
const [isInspectionModalOpen, setIsInspectionModalOpen] = useState(false);
const [isPaymentModalOpen, setIsPaymentModalOpen] = useState(false);
const [isRatingModalOpen, setIsRatingModalOpen] = useState(false);
const [isReportModalOpen, setIsReportModalOpen] = useState(false);
const [isInquiryModalOpen, setIsInquiryModalOpen] = useState(false);
const [paymentUserData, setPaymentUserData] = useState<{name,email,phone} | undefined>();

const [relatedPage, setRelatedPage] = useState(1);
```

```
const [showAllRelated, setShowAllRelated] = useState(false);
const [sameAgentOnly, setSameAgentOnly] = useState(false);
```

## 7.3 Image carousel

```
const nextImages = useCallback(() => {
  setCurrentIndex((prev) => Math.min(prev + step, Math.max(0, images.length - step)));
}, [images.length, step]);

const prevImages = useCallback(() => {
  setCurrentIndex((prev) => Math.max(prev - step, 0));
}, [step]);

// Detect portrait vs landscape per image so the layout can adapt
const handleImageLoad = (index: number, e: React.SyntheticEvent<HTMLImageElement>) => {
  const { naturalWidth, naturalHeight } = e.currentTarget;
  setOrientations((prev) => ({
    ...prev,
    [index]: naturalWidth >= naturalHeight ? "landscape" : "portrait",
  }));
};
```

Keyboard navigation handles both the carousel and the lightbox:

```
useEffect(() => {
  const handleKeyDown = (e: KeyboardEvent) => {
    if (isLightboxOpen) {
      if (e.key === "ArrowRight") nextLightboxImage();
      else if (e.key === "ArrowLeft") prevLightboxImage();
      else if (e.key === "Escape") setIsLightboxOpen(false);
    } else {
      if (e.key === "ArrowRight") nextImages();
      else if (e.key === "ArrowLeft") prevImages();
    }
  };
  window.addEventListener("keydown", handleKeyDown);
  return () => window.removeEventListener("keydown", handleKeyDown);
}, [nextImages, prevImages, nextLightboxImage, prevLightboxImage, isLightboxOpen]);
```

Touch swipe:

```
const [touchStart, setTouchStart] = useState<number | null>(null);
const [touchEnd, setTouchEnd] = useState<number | null>(null);
const minSwipeDistance = 50;

const onTouchEnd = () => {
  if (!touchStart || !touchEnd) return;
  const distance = touchStart - touchEnd;
  if (distance > minSwipeDistance) {        // swipe left -> next
    isLightboxOpen ? nextLightboxImage() : nextImages();
  } else if (distance < -minSwipeDistance) { // swipe right -> prev
    isLightboxOpen ? prevLightboxImage() : prevImages();
  }
};
```

## 7.4 Sharing

```
const handleShare = async () => {
  if (!property) return;
  const description = stripHtmlTags(property.description);
  const url = window.location.href;
  const address = `${property.location.area}, ${property.location.city_town}, ${property.location.state} state.`;
  const coverImageUrl = property.img || property.images?.[0] || "";
  const shareText = `${property.name}\n${address}\n\n${description}\n\nCategory: ${property.category}\nURL: ${url}`;

  if (navigator.share) {
    try {
      const shareData = { title: property.name, text: shareText };

      // Prefer the property video as the attached file...
      const videoUrl = property.videoUrl;
      if (videoUrl && typeof File !== "undefined") {
        try {
          const blob = await (await fetch(videoUrl)).blob();
          if (blob.size) {
            const shareFile = new File([blob], `${property.slug || property.name}.mp4`,
              { type: blob.type || "video/mp4" });
            if (!navigator.canShare || navigator.canShare({ files: [shareFile] })) {
              await navigator.share({ ...shareData, files: [shareFile] });
              return;
            }
          }
        } catch (videoError) { console.error("Error preparing share video:", videoError); }
      }

      // ...then the cover image...
      if (coverImageUrl && typeof File !== "undefined") {
        try {
          const blob = await (await fetch(coverImageUrl)).blob();
          if (blob.size) {
            const shareFile = new File([blob], `${property.slug || property.name}.jpg`,
              { type: blob.type || "image/jpeg" });
            if (!navigator.canShare || navigator.canShare({ files: [shareFile] })) {
              await navigator.share({ ...shareData, files: [shareFile] });
              return;
            }
          }
        } catch (imageError) { console.error("Error preparing share image:", imageError); }
      }

      await navigator.share(shareData);        // text only
    } catch (err) { console.error("Error sharing:", err); }
  } else {
    navigator.clipboard.writeText(`${shareText}`);
    toast.success("Link copied to clipboard!");
  }
};
```

## 7.5 Shortlist

```
const isShortlisted = property
  ? shortlistedProperties.some((p) => p.id === property.id)
  : false;

const handleShortlist = async () => {
  if (!property) return;
  const { isAuthenticated } = useAuthStore.getState();   // read outside React
  if (!isAuthenticated) {
    toast.error("Please log in to access your wishlist.");
```

```
    return;
  }
  toast.success(isShortlisted ? "Removed from shortlist" : "Added to shortlist");
  void toggleShortlist(property);
};
```

## 7.6 Video tour

```
{property.videoUrl.includes("youtube.com") || property.videoUrl.includes("youtu.be") ? (
  <iframe
    src={
      property.videoUrl.includes("youtube.com")
        ? property.videoUrl.replace("watch?v=", "embed/")
        : property.videoUrl.replace("youtu.be/", "youtube.com/embed/")
    }
    ...
  ></iframe>
) : (
  <video src={property.videoUrl} poster={property.img} controls preload="none" ... />
)}
```

YouTube watch URLs are rewritten to `/embed/` so the video can be embedded.

## 7.7 The sticky bottom action bar

```
<div
  className={`fixed bottom-0 left-0 right-0 z-40 bg-white/90 dark:bg-[#1A1A1A]/90
    backdrop-blur-md border-t ... transition-transform duration-300 ease-in-out
    ${showActionBar ? "translate-y-0" : "translate-y-full"}`}
>
  <div className="max-w-6xl mx-auto">
    {/* price + visitation fee summary */}
    <div className="flex items-center justify-between gap-3 mb-2 pb-2 border-b ...">
      <div>
        <p className="text-[10px] uppercase tracking-wide ...">Price {property?.duration && `(${property.duration})`}</p>
        <p className="text-base md:text-lg font-bold text-[#703BF7]">₦{formatCurrency(price)}</p>
      </div>
      {property?.visitationfee ? (
        <div className="text-right">
          <p className="text-[10px] uppercase ...">Visitation Fee</p>
          <p className="text-sm md:text-base font-semibold">₦{formatCurrency(property.visitationfee)}</p>
        </div>
      ) : null}
    </div>

    {/* 4 buttons: Inquire | Book a Visit | Make Payment | Report Agent */}
    <div className="grid grid-cols-2 md:grid-cols-4 gap-3"> ... </div>
  </div>
</div>
```

It slides in on scroll-down and auto-hides after 5 s:

```
useEffect(() => {
  const handleScroll = () => {
    const currentY = window.scrollY;
    if (currentY > lastScrollY.current) setShowActionBar(true);
    else setShowActionBar(false);
    lastScrollY.current = currentY;

    if (hideTimeoutRef.current) clearTimeout(hideTimeoutRef.current);
    hideTimeoutRef.current = setTimeout(() => setShowActionBar(false), 5000);
```

```
  };
  window.addEventListener("scroll", handleScroll, { passive: true });
  return () => {
    window.removeEventListener("scroll", handleScroll);
    if (hideTimeoutRef.current) clearTimeout(hideTimeoutRef.current);
  };
}, []);
```

## 7.8 Body scroll lock while a modal is open

```
useEffect(() => {
  if (isInspectionModalOpen || isPaymentModalOpen || isInquiryModalOpen) {
    document.body.style.overflow = "hidden";
  } else {
    document.body.style.overflow = "";
  }
  return () => { document.body.style.overflow = ""; };
}, [isInspectionModalOpen, isPaymentModalOpen, isInquiryModalOpen]);
```

## 7.9 Related properties

```
const RELATED_ITEMS_PER_PAGE = 4;
const totalRelatedPages = Math.ceil(totalRelatedProperties / RELATED_ITEMS_PER_PAGE);

useEffect(() => {
  if (property) {
    const limit = showAllRelated ? 100 : RELATED_ITEMS_PER_PAGE;
    const page = showAllRelated ? 1 : relatedPage;
    fetchRelatedProperties(property, sameAgentOnly, page, limit);
  }
}, [property, sameAgentOnly, relatedPage, showAllRelated, fetchRelatedProperties]);

useEffect(() => {
  setRelatedPage(1);
  setShowAllRelated(false);
}, [sameAgentOnly, property?.id]);
```

## 7.10 The modals it owns

```
<BookInspectionModal property={property} open={isInspectionModalOpen}
  onOpenChange={setIsInspectionModalOpen} />

<PropertyPaymentModal property={property} open={isPaymentModalOpen}
  onOpenChange={setIsPaymentModalOpen}
  onPaymentSuccess={(userData) => {
    setPaymentUserData(userData);
    setIsRatingModalOpen(true);        // -> ask for a rating after paying
  }} />

<ServiceRatingModal property={property} open={isRatingModalOpen}
  onOpenChange={setIsRatingModalOpen} paymentUserData={paymentUserData} />

<ReportAgentModal property={property} open={isReportModalOpen}
  onOpenChange={setIsReportModalOpen} />

<InquiryModal property={property} open={isInquiryModalOpen}
  onOpenChange={setIsInquiryModalOpen} />
```

# 8. Properties: shared components

`src/features/properties/components/` contains 15 `.tsx` components plus `propertyFeatureIcons.ts`.

| File | Purpose |
|---|---|
| `PropertyCard.tsx` | The card used in every grid (listing, related, featured) |
| `PropertySection.tsx` | "Featured Properties" strip used on the home page |
| `PropertyFeatureCard.tsx` | Tile with an icon, used for amenities / rules / bills |
| `PropertyFeatureItem.tsx` | Row variant of the same, used in the sidebar |
| `FeatureSection.tsx` | Groups feature cards + renders the "Show all …" button |
| `propertyFeatureIcons.ts` | Label → icon lookup table (44 entries) |
| `CustomDropdown.tsx` | Hand-rolled select used by the filter bar |
| `PropertyMap.tsx` | Leaflet map with price markers and fullscreen mode |
| `PropertyFiltersModal.tsx` | Full-screen filter sheet (hand-rolled, not Radix) |
| `PropertyPaymentModal.tsx` | Buy a property (Paystack) |
| `CoRentModal.tsx`, `InquiryModal.tsx`, `ListPropertyModal.tsx`, `PreferenceModal.tsx`, `ReportAgentModal.tsx`, `ServiceRatingModal.tsx` | Radix dialogs — see §9 |

## 8.1 `PropertyCard.tsx`

The card owns a mini image carousel with a sliding "window" of dots.

```
const DOTS_PER_GROUP = 4;
const DOT_SIZE = 6;
const DOT_GAP = 6;
const SWIPE_THRESHOLD = 40;

const images: string[] =
  Array.isArray((property as Property).images) && (property as Property).images.length > 0
    ? (property as Property).images
    : [property.img];

const [currentIndex, setCurrentIndex] = useState(0);
const hasMultipleImages = images.length > 1;

// Show at most 4 dots, centred on the active image
const windowSize = Math.min(DOTS_PER_GROUP, images.length);
const windowStart = Math.max(
  0,
  Math.min(currentIndex - Math.floor(windowSize / 2), images.length - windowSize),
);

const goToIndex = (index: number) => {
  setCurrentIndex(Math.max(0, Math.min(index, images.length - 1)));
};
```

The carousel is a CSS translate, not a scroll container:

```
<div
  className="flex transition-transform duration-500 ease-in-out touch-pan-y"
  style={{ transform: `translateX(-${currentIndex * 100}%)` }}
>
  {images.map((img, idx) => (
    <img key={idx} src={img} alt={`${property.name} ${idx + 1}`}
      loading="lazy" decoding="async" draggable={false}
      className="w-full h-44 object-cover shrink-0 bg-gray-200 dark:bg-neutral-800" />
  ))}
```

```
</div>
```

Dots are translated so the active one stays visible:

```
<div className="absolute z-10 bottom-2 left-1/2 -translate-x-1/2 overflow-hidden"
  style={{ width: `${windowSize * DOT_SIZE + (windowSize - 1) * DOT_GAP}px` }}>
  <div className="flex items-center transition-transform duration-300 ease-out"
    style={{ gap: `${DOT_GAP}px`, transform: `translateX(-${windowStart * (DOT_SIZE + DOT_GAP)}px)` }}>
    {images.map((_, idx) => (
      <button key={idx} onClick={(e) => handleDotClick(idx, e)}
        className={`rounded-full shrink-0 transition-colors duration-300 cursor-pointer w-1.5 h-1.5 ${idx === currentIndex
↪  ? "bg-white" : "bg-white/50 hover:bg-white/80"}`} />
    ))}
  </div>
</div>
```

Swipe detection distinguishes a horizontal swipe from a vertical scroll:

```
const touchStartRef = useRef({ x: 0, y: 0 });
const isSwipingRef = useRef(false);
const justSwipedRef = useRef(false);

const handleTouchMove = (e: React.TouchEvent) => {
  const touch = e.touches[0];
  const deltaX = touch.clientX - touchStartRef.current.x;
  const deltaY = touch.clientY - touchStartRef.current.y;
  if (Math.abs(deltaX) > Math.abs(deltaY) && Math.abs(deltaX) > 10) {
    isSwipingRef.current = true;
  }
};

const handleTouchEnd = (e: React.TouchEvent) => {
  if (!hasMultipleImages) return;
  const deltaX = e.changedTouches[0].clientX - touchStartRef.current.x;
  if (isSwipingRef.current && Math.abs(deltaX) > SWIPE_THRESHOLD) {
    e.preventDefault();
    e.stopPropagation();
    justSwipedRef.current = true;                 // suppress the click that follows
    if (deltaX < 0) goToIndex(currentIndex + 1);
    else goToIndex(currentIndex - 1);
  }
  isSwipingRef.current = false;
};
```

Because a swipe also fires a click, the click handler checks the flag:

```
const handleCardClick = () => {
  if (justSwipedRef.current) {
    justSwipedRef.current = false;
    return;
  }
  navigate(`/properties/${propertySlug}`);
};
```

Shortlisting from the card (same auth guard as the detail page):

```
const isShortlisted = shortlistedProperties.some((p) => p.id === property.id);

const handleShortlist = async (e: React.MouseEvent) => {
  e.preventDefault();
  e.stopPropagation();

  const { isAuthenticated } = useAuthStore.getState();
  if (!isAuthenticated) {
```

```
    toast.error("Please log in to access your wishlist.");
    return;
  }
  toast.success(isShortlisted ? "Removed from shortlist" : "Added to shortlist");
  void toggleShortlist(property);
};
```

The listing-type badge is coloured by a regex chain:

```
{property.listingType && (
  <span className={`text-[10px] font-bold uppercase tracking-wider px-2 py-1 rounded-sm shadow
↪  ${/sale/i.test(property.listingType)
      ? "bg-emerald-500 text-white"
      : /shortlet/i.test(property.listingType)
        ? "bg-rose-500 text-white"
        : /rent/i.test(property.listingType)
          ? "bg-[#703BF7] text-white"
          : /lease/i.test(property.listingType)
            ? "bg-amber-500 text-white"
            : /invest/i.test(property.listingType)
              ? "bg-sky-500 text-white"
              : "bg-black/60 text-white"}`}>
    {property.listingType}
  </span>
)}
```

The map link uses lat/lng when present and falls back to a text search:

```
href={
  property.geo_location?.lat !== 0 && property.geo_location?.lat !== null &&
  property.geo_location?.lng !== 0 && property.geo_location?.lng !== null
    ? `https://www.google.com/maps/search/?api=1&query=${property.geo_location.lat},${p roperty.geo_location.lng}`
    : `https://www.google.com/maps/search/?api=1&query=${encodeURIComponent(
        [property.location.area, property.location.city_town || property.location.city, property.location.state]
          .filter(Boolean).join(", ") + (property.location.state ? " state." : ""),
      )}`
}
```

Descriptions arrive with HTML from the API, so they are stripped and truncated:

```
function truncateWords(text: string, limit: number): string {
  const plain = text.replace(/<[^>]+>/g, " ").replace(/\s+/g, " ").trim();
  const words = plain.split(" ");
  return words.length <= limit ? plain : `${words.slice(0, limit).join(" ")}...`;
}
```

## 8.2 `propertyFeatureIcons.ts`

44 labelled icons in 3 groups (26 offers / 13 rules / 5 bills) plus 6 aliases.

```
export const propertyFeatureIconGroups: PropertyFeatureIconGroup[] = [
  {
    title: "What this place offers",
    description: "Features and amenities available at the property.",
    items: [
      { label: "Borehole", icon: Drill },
      { label: "Water heater", icon: ThermometerSun },
      { label: "Solar panel", icon: Sun },
      { label: "Swimming pool", icon: Waves },
      { label: "Generator", icon: Fuel },
      { label: "Air conditioning", icon: AirVent },
      { label: "CCTV", icon: Cctv },
```

```
      { label: "Wi-Fi", icon: Wifi },
      { label: "Parking space", icon: SquareParking },
      { label: "Inverter", icon: BatteryCharging },
      { label: "Wardrobes", icon: Shirt },
      { label: "En-suite bathroom", icon: Bath },
      { label: "Gym", icon: Dumbbell },
      { label: "Balcony", icon: DoorOpen },
      { label: "Garden", icon: Flower2 },
      { label: "24-hour security", icon: ShieldCheck },
      { label: "24-hour electricity supply", icon: Zap },
      { label: "POP ceiling", icon: LampCeiling },
      { label: "Elevator", icon: ArrowUpDown },
      { label: "Boys' quarters (BQ)", icon: House },
      { label: "Fitted kitchen", icon: CookingPot },
      { label: "Fenced compound", icon: Fence },
      { label: "Intercom system", icon: PhoneCall },
      { label: "Tiled floors", icon: LayoutGrid },
      { label: "Running water", icon: Droplets },
      { label: "Personal prepaid meter", icon: Gauge },
    ],
  },
  {
    title: "House rules",
    items: [
      { label: "No pets allowed", icon: PawPrint, prohibited: true },
      { label: "No noise", icon: VolumeX },
      { label: "Female only", icon: Venus },
      { label: "Men only", icon: Mars },
      { label: "Married couples only", icon: Heart },
      { label: "No cooking in rooms", icon: CookingPot, prohibited: true },
      { label: "No subletting", icon: KeyRound, prohibited: true },
      { label: "No parties or events", icon: PartyPopper, prohibited: true },
      { label: "No unauthorized alterations", icon: Hammer, prohibited: true },
      { label: "No unauthorized visitors", icon: UserX },
      { label: "Visitation time applies", icon: Clock },
      { label: "Visitors not allowed", icon: Users, prohibited: true },
      { label: "No smoking", icon: CigaretteOff },
    ],
  },
  {
    title: "Bills & fees",
    items: [
      { label: "Electric bill", icon: Zap },
      { label: "Water bill", icon: Droplet },
      { label: "Waste disposal fee", icon: Trash2 },
      { label: "Security fee", icon: ShieldCheck },
      { label: "Service charge", icon: Receipt },
    ],
  },
];
```

Lookup normalises the label (lowercase, alphanumeric only) and resolves aliases:

```
const normalizeFeatureLabel = (label: string) =>
  label.toLowerCase().replace(/[^a-z0-9]/g, "");

const propertyFeatureByLabel = new Map(
  propertyFeatureIconGroups.flatMap((group) =>
    group.items.map((item) => [normalizeFeatureLabel(item.label), item]),
  ),
);

const featureLabelAliases = new Map([
  ["mariedcouplesonly", "marriedcouplesonly"],
  ["noloudmusicorexcessivenoise", "nonoise"],
  ["noexcessivenoise", "nonoise"],
  ["nosmokinginsidetheproperty", "nosmoking"],
```

```
  ["visitorsarenotallowed", "visitorsnotallowed"],
  ["unauthorizedvisitorsnotallowed", "nounauthorizedvisitors"],
]);

export const getPropertyFeatureItem = (label: string) => {
  const normalizedLabel = normalizeFeatureLabel(label);
  const canonicalLabel = featureLabelAliases.get(normalizedLabel) ?? normalizedLabel;
  return propertyFeatureByLabel.get(canonicalLabel);
};
```

## 8.3 `PropertyFeatureCard.tsx` / `PropertyFeatureItem.tsx`

Both read the same lookup table. The card is a grid tile, the item is a list row.

```
// PropertyFeatureCard
const feature = getPropertyFeatureItem(label);
const Icon = feature?.icon ?? House;
const isProhibited = variant === "rule" && (feature?.prohibited ?? false);

<li className={`relative flex min-h-28 flex-col items-center justify-center gap-2 rounded-xl border p-4 text-center ${
    isProhibited ? "border-red-500/40 bg-red-500/5" : "border-gray-600/20 bg-gray-50 dark:bg-black/20"}`}>
  <span className="relative text-[#703BF7]">
    <Icon aria-hidden="true" size={24} strokeWidth={1.5} />
    {isProhibited && (
      <span className="absolute -right-2 -top-2 flex h-4 w-4 items-center justify-center rounded-full border border-white
↪  bg-red-600 text-white dark:bg-[#1A1A1A]">
        <Ban aria-hidden="true" size={10} strokeWidth={2} />
      </span>
    )}
  </span>
  <span className="text-sm text-gray-700 dark:text-gray-300">{label}</span>
</li>
```

```
// PropertyFeatureItem
const feature = getPropertyFeatureItem(label);
const Icon = icon ?? feature?.icon ?? (showIcon ? House : null);
const isProhibited = variant === "rule" && (feature?.prohibited ?? false);

<li className={`flex items-center gap-3 py-3 text-gray-700 dark:text-gray-300 ${
    divided ? "border-b border-gray-200 last:border-b-0 dark:border-white/10" : ""} ${
    highlighted ? "gap-0 rounded-md border-l border-[#703BF7] bg-[#703BF7]/5 p-2" : ""}`}>
  {Icon && (
    <span className={`relative shrink-0 ${isProhibited ? "text-red-600" : highlighted ? "text-[#703BF7]" : "text-gray-700
↪  dark:text-gray-300"}`}>
      <Icon aria-hidden="true" size={20} strokeWidth={1.6} className={highlighted ? "fill-[#703BF7]" : undefined} />
      {isProhibited && (
        <span className="absolute -right-1.5 -top-1.5 flex h-3.5 w-3.5 items-center justify-center rounded-full bg-red-600
↪  text-white">
          <Ban aria-hidden="true" size={9} strokeWidth={2} />
        </span>
      )}
    </span>
  )}
  <span className="text-sm leading-6">{label}</span>
</li>
```

## 8.4 `FeatureSection.tsx`

Renders a titled grid of feature cards, truncated to `limit`, with a ”Show all …” button that opens the corresponding modal.

```
export default function FeatureSection({ title, items = [], extraItems = [], limit, noun, onShowAll, variant }:
↪  FeatureSectionProps) {
  if (items.length === 0 && extraItems.length === 0) return null;

  const hasMore = items.length > limit || extraItems.length > 0;

  return (
    <section className="border-t border-gray-200 py-7 first:border-t-0 first:pt-0 dark:border-white/10">
      <h2 className="mb-4 text-xl font-semibold tracking-tight text-gray-900 dark:text-white">{title}</h2>
      <ul className="grid grid-cols-2 gap-3 sm:grid-cols-3">
        {items.slice(0, limit).map((item, index) => (
          <PropertyFeatureCard key={`${item}-${index}`} label={item} variant={variant} />
        ))}
      </ul>
      {hasMore && (
        <button type="button" onClick={onShowAll}
          className="mt-5 rounded-lg border border-gray-900 px-4 py-2.5 text-sm font-semibold ...">
          Show all {noun}
        </button>
      )}
    </section>
  );
}
```

## 8.5 `CustomDropdown.tsx`

A native-`<select>` replacement. Closes on outside `mousedown`.

```
const [open, setOpen] = useState(false);
const ref = useRef<HTMLDivElement>(null);

useEffect(() => {
  const handleClick = (e: MouseEvent) => {
    if (!ref.current?.contains(e.target as Node)) setOpen(false);
  };
  document.addEventListener("mousedown", handleClick);
  return () => document.removeEventListener("mousedown", handleClick);
}, []);

const selectedOption = options.find((o) => o.value === value);
const isSelected = Boolean(value && selectedOption && selectedOption.value !== "");
const selectedLabel = selectedOption ? selectedOption.label : placeholder;
```

The menu is always mounted and animated with opacity/transform, and is inert when closed:

```
<div className={`absolute z-50 mt-2 min-w-full w-full rounded-xl border border-gray-700 bg-gray-300 dark:bg-neutral-900
↪  shadow-xl overflow-hidden left-0 transition-all duration-200 ease-out origin-top transform ${
    open ? "opacity-100 scale-y-100 translate-y-0 pointer-events-auto"
         : "opacity-0 scale-y-95 -translate-y-2 pointer-events-none"}`}
  style={{ zIndex: 9999 }}>
  <div className="max-h-72 overflow-y-auto overflow-x-auto">
    {options.map((option) => (
      <button key={option.value} type="button"
        onClick={() => { onChange(option.value); setOpen(false); }}
        className={`block w-full px-4 py-3 text-left whitespace-nowrap transition text-sm ${
          value === option.value ? "bg-[#703BF7] text-white font-medium"
                                 : "hover:bg-gray-200 dark:hover:bg-neutral-800 ..."}`}>
        {option.label}
      </button>
```

```
    ))}
  </div>
</div>
```

## 8.6 `PropertyMap.tsx`

Leaflet map, OpenStreetMap tiles, one price-pill marker per property.

```
function getCoords(p: Property): [number, number] | null {
  const lat = p.geo_location?.lat;
  const lng = p.geo_location?.lng;
  if (typeof lat === "number" && typeof lng === "number" && !(lat === 0 && lng === 0)) {
    return [lat, lng];
  }
  return null;                        // 0,0 means "no coordinates"
}

function formatPrice(price?: number): string {
  if (!price) return "₦—";
  if (price >= 1_000_000_000) return `₦${(price / 1_000_000_000).toFixed(1)}B`;
  if (price >= 1_000_000) return `₦${(price / 1_000_000).toFixed(1)}M`;
  if (price >= 1_000) return `₦${(price / 1_000).toFixed(0)}K`;
  return `₦${price}`;
}
```

Markers are `divIcon`s with inline HTML:

```
function makePriceIcon(label: string, active: boolean) {
  return L.divIcon({
    className: "property-price-marker",
    html: `
      <div style="background:${active ? "#5c2fe0" : "#e11d48"};
        color:#fff; font-weight:600; font-size:12px; padding:4px 8px;
        border-radius:999px; white-space:nowrap;
        box-shadow:0 2px 6px rgba(0,0,0,0.35); border:2px solid #fff;
        transform:${active ? "scale(1.12)" : "scale(1)"}; transition:transform .15s ease;">
        ${label}
      </div>`,
    iconAnchor: [20, 14],
  });
}
```

`FitBounds` auto-fits only when the actual set of points changes. This is deliberate — it used to snap back on every parent re-render, making markers impossible to click:

```
function FitBounds({ points, pointsKey }: { points: [number, number][]; pointsKey: string }) {
  const map = useMap();
  const lastFitKeyRef = useRef<string | null>(null);

  useEffect(() => {
    if (points.length === 0) return;
    if (lastFitKeyRef.current === pointsKey) return;   // already fitted for this exact set
    lastFitKeyRef.current = pointsKey;

    if (points.length === 1) map.setView(points[0], 12);
    else map.fitBounds(L.latLngBounds(points), { padding: [40, 40] });
  }, [pointsKey, map]);
  return null;
}
```

The key is derived from the coordinate content, not the array identity:

```
const pointsKey = useMemo(
  () => pinned.map(({ property, coords }) => `${property.id}:${coords[0]},${coords[1]}`).join("|"),
  [pinned],
);
```

Fullscreen mode is portalled to `document.body` so it escapes ancestor stacking contexts, and locks page scroll:

```
if (isFullscreen && typeof document !== "undefined") {
  return (
    <>
      <div className={`${heightClassName} w-full`} aria-hidden="true" />
      {createPortal(
        <div className="fixed inset-0 z-9999 w-full h-full bg-white dark:bg-[#111] p-2 sm:p-4">
          <div className="relative w-full h-full rounded-2xl overflow-hidden border ...">
            {mapBody}
            <ExpandButton isFullscreen onToggle={() => setIsFullscreen(false)} visibilityClassName="flex" />
          </div>
        </div>,
        document.body,
      )}
    </>
  );
}
```

Marker hover/click are pushed back to the parent, so hovering a marker can highlight the matching card in the list:

```
eventHandlers={{
  mouseover: () => onHoverProperty?.(property.id),
  mouseout: () => onHoverProperty?.(null),
  click: () => onSelectProperty?.(property.id),
}}
```

## 8.7 `PropertySection.tsx`

The home-page ”Featured Properties” strip (4 cards).

```
const { featuredProperties, featuredLoading, error, fetchFeaturedProperties } = usePropertyStore();
useEffect(() => { fetchFeaturedProperties(); }, [fetchFeaturedProperties]);

const currentProperties = featuredProperties.slice(0, 4);

// The 4th card is hidden between sm and xl (only 3 fit on md)
const hideOnMdOnly = "hidden sm:block md:hidden xl:block";
```

Loading, error and success states are mutually exclusive:

```
{featuredLoading ? (
  [...Array(4)].map((_, i) => (
    <div key={i} className={`${i === 3 ? hideOnMdOnly : ""} h-full`}>
      <PropertyCardSkeleton />
    </div>
  ))
) : error ? (
  <div className="col-span-1 sm:col-span-2 md:col-span-3 lg:col-span-4 py-10 ...">
    ... Something went wrong ... <button onClick={() => fetchFeaturedProperties()}>Try Again</button>
  </div>
) : (
  currentProperties.map((item, i) => (
    <div key={item.id} className={`${i === 3 ? hideOnMdOnly : ""} h-full`}>
      <PropertyCard property={item} />
    </div>
  ))
```

```
)}
```

# 9. Properties: modals

Eight property modals live in `src/features/properties/components/`. Seven of them use Radix Dialog; `PropertyFiltersModal` is hand-rolled.

| Modal | Trigger | What it does |
|---|---|---|
| `PropertyPaymentModal` | ”Make Payment” on the detail page | Full Paystack purchase flow |
| `BookInspectionModal` | ”Book a Visit” | Pays the visitation fee, books a visit |
| `InquiryModal` | ”Inquire” | Sends a message to the agent |
| `ServiceRatingModal` | After a successful payment | Star rating + Trustpilot invite |
| `ReportAgentModal` | ”Report Agent” | Reports a listing to the CRM |
| `PreferenceModal` | ”Prefered Location” / home page | Registers a property search brief |
| `CoRentModal` | ”Co-rent” | Registers a co-tenant match request |
| `ListPropertyModal` | ”List a Property” | Registers a lister application |
| `PropertyFiltersModal` | ”Filters” button on the listing page | Mobile/desktop filter sheet |

## 9.1 The shared pattern

Every lead-capture modal follows the same shape:

1. Read a saved form from a cookie so a user does not lose their typing.
2. If the user is signed in, pull the name/email/phone from `authAPI.getProfile()`.
3. Validate locally.
4. `POST https://api.sabiflow.com/api/crm/deals/guest` with a CRM payload.
5. Clear the form + cookie and toast the result.

Cookie helpers come from `src/shared/lib/utils.ts`:

```
const inquiryCookieKey = "rewaciti_property_details_inquiry";
const cookieLoaded = useRef(false);

useEffect(() => {
  const savedInquiry = getCookie(inquiryCookieKey);
  if (savedInquiry) {
    try {
      const parsed = JSON.parse(savedInquiry);
      setFullName(parsed.fullName || "");
      setEmail(parsed.email || "");
      setPhone(parsed.phone || "");
      setMessage(parsed.message || "");
      setAgreed(parsed.agreed || false);
    } catch {
      // ignore malformed cookie data
    }
  }
  ...
}, [open, isAuthenticated, customer]);

useEffect(() => {
  if (!cookieLoaded.current) {          // don't overwrite the cookie on first paint
    cookieLoaded.current = true;
    return;
  }
  setCookie(inquiryCookieKey, JSON.stringify({ fullName, email, phone, message, agreed }));
}, [fullName, email, phone, message, agreed]);
```

The CRM payload shape is consistent:

```
const payload = {
  companyId: COMPANY_ID,
  pipelineId: "69b49c7541d35d158e336621",
  title: `${trimmedName} interested in ${property.name}`,
  name: trimmedName,
  amount: price,
  email,
  phone,
  address: `${property.location.area}, ${property.location.city_town}, ${property.location.state} state.`,
  note: message,
  ...(ownerId ? { ownerId } : {}),
  customData: [
    { label: "Property Name", value: property.name },
    { label: "Property Link", value: propertyUrl },
    { label: "Category", value: property.category },
    { label: "Location", value: "..." },
    ...(property.createdBy?.email ? [{ label: "Agent Email", value: property.createdBy.email }] : []),
    ...(property.sku ? [{ label: "Property SKU", value: property.sku }] : []),
    ...(property.caretakerContact?.whatsapp ? [{ label: "Caretaker WhatsApp", value: property.caretakerContact.whatsapp }]
↪  : []),
    ...(property.caretakerContact?.phone ? [{ label: "Caretaker Phone", value: property.caretakerContact.phone }] : []),
  ],
};

await axios.post("https://api.sabiflow.com/api/crm/deals/guest", payload);
```

## 9.2 `PropertyPaymentModal.tsx` — buying a property

This is the only modal that talks to the payments API.

```
const API_URL = import.meta.env.VITE_API_URL;
const API_KEY = import.meta.env.VITE_SABIFLOW_API_KEY;
const GATEWAY_ID = import.meta.env.VITE_SABIFLOW_GATEWAY_ID;
```

Fee logic — an agent-owned property charges a service fee instead of an agent fee:

```
const price = property.pricing.TotalCost || 0;
const pricing = property.pricing;
const isAgentOwned =
  (property.creatorClassification ?? property.createdBy?.classification ?? "")
    .toLowerCase() === "agent";
const deductionAmount = isAgentOwned ? pricing.ServiceFee : pricing.AgentFee;
const deductionLabel = isAgentOwned ? "Service Charge" : "Agent Fee";
const netPayable = price - deductionAmount;
```

Guests and signed-in users see different forms; the same state feeds both:

```
const fullName = customer ? `${customer.firstName} ${customer.lastName}`.trim() : guestName;
const email = customer?.email ?? guestEmail;
const phone = profilePhone || customer?.phoneNumber || guestPhone;
```

Validation, including the ₦5M ceiling:

```
if (!fullName.trim() || !email.trim() || !phone.trim()) {
  toast.error("Please enter your full name, email, and phone number to continue.");
  return;
}
if (!agreed) {
  toast.error("Please agree to the Terms and Privacy Policy to continue.");
  return;
}
if (price > 5000000) {
```

```
  toast.warning("For payments above ₦5,000,000, please contact our support team directly for secure bank transfer
↪  options.", { duration: 6000 });
  return;
}
```

**Step 1 — create the sale:**

```
const salePayload = {
  initiateInvoice: true,
  gatewayId: GATEWAY_ID,
  items: [
    {
      description: `Property Payment: ${property.name}, ${propertyAddress}`,
      quantity: 1,
      unitPrice: price,
      inventoryItemId: property.id,
      productId: property.id,
    }
  ],
  taxRate: 0,
  notes: `Payment for ${property.name} at ${propertyAddress}. Property Link: ${propertyUrl}`,
  paymentMethod: "credit_card",
  customerDetails: { name: fullName, email: email, phone: phone }
};

const saleResponse = await axios.post<SaleResponse>(`${API_URL}/sales`, salePayload, {
  headers: { "x-api-key": API_KEY }
});

const saleId = saleResponse.data.id || saleResponse.data._id;
if (!saleId) throw new Error("Sale creation response did not include a sale ID.");
```

**Step 2 — initialise the payment and get an access code:**

```
const initResponse = await axios.post(`${API_URL}/sales/${saleId}/payment/initiate`, {
  gatewayId: GATEWAY_ID,
  initiateGateway: true,
}, { headers: { "x-api-key": API_KEY } });

const accessCode = initResponse.data.data.accessCode;
```

**Step 3 — open the Paystack modal** (`@paystack/inline-js`, not `react-paystack`):

```
const paystack = new PaystackPop();
paystack.resumeTransaction(accessCode, {
  onSuccess: async () => { ... },
  onCancel: () => {
    toast.info("Payment cancelled.");
    setIsSubmitting(false);
  }
});
```

**Step 4 — verify, then push to the CRM:**

```
await axios.post(`${API_URL}/sales/${saleId}/payment/verify`, {
  gatewayId: GATEWAY_ID
}, { headers: { "x-api-key": API_KEY } });

const crmPayload = {
  companyId: COMPANY_ID,
  pipelineId: "69ed7a4ac09e9388ba096f1f",
  inventoryItemId: property.id,
  title: `PROPERTY PAYMENT: ${property.name} - ${fullName}`,
  name: fullName,
```

```
  amount: price,
  email, phone,
  address: propertyAddress,
  note: `Payment of property for ${property.name} by ${fullName}.`,
  customData: [
    { label: "Property Name", value: property.name },
    { label: "Property Link", value: propertyUrl },
    { label: "amount Paid", value: `₦${price.toLocaleString()}` },
    { label: deductionLabel, value: `₦${deductionAmount.toLocaleString()}` },
    { label: "Net Payable", value: `₦${netPayable.toLocaleString()}` },
    ...
  ]
};

await axios.post("https://api.sabiflow.com/api/crm/deals/guest", crmPayload);
```

**Step 5 — download the receipt and hand over to the rating modal:**

```
const handleDownloadReceipt = async (saleId: string) => {
  const response = await axios.get(`${API_URL}/sales/${saleId}/invoice/download?format=pdf`, {
    headers: { "x-api-key": API_KEY },
    responseType: 'blob'
  });
  const url = window.URL.createObjectURL(new Blob([response.data]));
  const link = document.createElement('a');
  link.href = url;
  link.setAttribute('download', `receipt-${saleId}.pdf`);
  document.body.appendChild(link);
  link.click();
  link.remove();
};

await handleDownloadReceipt(saleId);
onOpenChange(false);
setCookie(modalCookieKey, JSON.stringify({ guestName: "", guestEmail: "", guestPhone: "", agreed: false }));

if (onPaymentSuccess) {
  onPaymentSuccess({ name: fullName, email: email, phone: phone });
}
```

If the sale cannot be created because the property was taken, the API message is turned into a friendly toast:

```
if (axios.isAxiosError(error)) {
  const errorMessage = error.response?.data?.message || "";
  if (errorMessage.toLowerCase().includes("stock") || errorMessage.toLowerCase().includes("available")) {
    toast.error("This property is no longer available. Please check out our other listings.", { duration: 6000 });
  } else {
    toast.error(errorMessage || "An error occurred during the payment process.");
  }
}
```

## 9.3 `ServiceRatingModal.tsx` — rating + Trustpilot

Opened automatically by the detail page after `onPaymentSuccess`.

```
declare global {
  interface Window {
    tp?: (...args: unknown[]) => void;
  }
}

// Fires the Trustpilot review invitation. Sent for every submission
// regardless of star rating — Trustpilot's guidelines prohibit only
```

```
// inviting customers who gave a high internal rating ("review gating").
const sendTrustpilotInvitation = () => {
  if (!userData?.email) return;

  window.tp?.("createInvitation", {
    recipientEmail: userData.email,
    recipientName: userData.name || "Customer",
    referenceId: property.id,
    source: "InvitationScript",
  });
};
```

The rating goes to the CRM as a separate deal:

```
const feedbackPayload = {
  companyId: COMPANY_ID,
  pipelineId: "6a27f05df19944ca7f9a87dd",
  title: `FEEDBACK: ${property.name}${userData ? ` - ${userData.name}` : ""}`,
  name: userData?.name || "Anonymous",
  email: userData?.email || "",
  phone: userData?.phone || "",
  amount: 0,
  note: `Rating: ${rating}/5\nFeedback: ${feedback}`,
  customData: [
    { label: "Property Name", value: property.name },
    { label: "Service Rating", value: `${rating} Stars` },
    { label: "Feedback", value: feedback || "No additional feedback provided" },
    ...(property.sku ? [{ label: "Property SKU", value: property.sku }] : [])
  ]
};

await axios.post("https://api.sabiflow.com/api/crm/deals/guest", feedbackPayload);
sendTrustpilotInvitation();
```

Closing the dialog navigates back to the listing page:

```
const handleClose = (nextOpen: boolean) => {
  onOpenChange(nextOpen);
  if (!nextOpen) navigate("/properties");
};
```

## 9.4 `ReportAgentModal.tsx` — fallback endpoints

This is the only modal that tries multiple CRM URLs.

```
const apiBaseUrl =
  import.meta.env.VITE_API_URL || "https://api.sabiflow.com/api";
const candidateUrls = [
  `${apiBaseUrl}/crm/deals/guest`,
  `${apiBaseUrl}/crm/deals`,
  "https://api.sabiflow.com/api/crm/deals/guest",
];

try {
  let lastError: unknown;

  for (const url of candidateUrls) {
    try {
      await axios.post(url, payload);
      toast.success("Report submitted successfully...");
      onOpenChange(false);
      setReason("");
      setDescription("");
```

```
      return;
    } catch (error) {
      lastError = error;
      const isMissingEndpoint =
        axios.isAxiosError(error) && error.response?.status === 404;
      if (!isMissingEndpoint) throw error;   // only fall through on 404
    }
  }

  throw lastError;
}
```

## 9.5 `PreferenceModal.tsx` — property search brief

```
const payload = {
  companyId: COMPANY_ID,
  pipelineId: "69b49c7541d35d158e336621",
  title: `Property Preference from ${name.trim()}`,
  name: name.trim(),
  email: email.trim(),
  phone: phone.trim(),
  address: preferedLocation.trim(),
  note: message || "No additional details added",
  customData: [
    { label: "Budget", value: budget },
    { label: "Category", value: preferedCategory },
    { label: "Preferred Location", value: preferedLocation.trim() },
    { label: "Bedrooms", value: bedroomsContact },
    { label: "Bathrooms", value: bathroomsContact },
    { label: "Move-in Timeline", value: moveInTimeline },
  ],
};

await axios.post("https://api.sabiflow.com/api/crm/deals/guest", payload);
```

## 9.6 `CoRentModal.tsx` — co-tenant matching

```
const payload = {
  companyId: COMPANY_ID,
  pipelineId: "6a9a70d121202229fbb92a98",
  title: `Co-Rent Request: ${name.trim()} (${gender || "Unspecified"} looking for ${preferredPartnerGender}) -
↪  ${preferedLocation.trim()}`,
  name: name.trim(),
  email: email.trim(),
  phone: phone.trim(),
  address: preferedLocation.trim(),
  note: message || "Looking to co-rent a property",
  customData: [
    { label: "Request Type", value: "Co-Rent Match" },
    { label: "Preferred Location", value: preferedLocation.trim() },
    { label: "Applicant Gender", value: gender || "Not specified" },
    { label: "Preferred Co-Rent Partner Gender", value: preferredPartnerGender },
    { label: "Room / Share Type", value: roomType || "Any" },
    { label: "Budget per Person", value: budget || "Flexible" },
    { label: "Move-in Timeline", value: moveInTimeline || "Flexible" },
    { label: "Lifestyle / About Applicant", value: message || "No extra notes" },
  ],
};
```

## 9.7 `ListPropertyModal.tsx` — become a lister

```
const payload = {
  companyId: COMPANY_ID,
  pipelineId: "6a9eb52b21202229fbb9eb00",
  title: `Become a Lister Request from ${name.trim()} (${listerRole}) - ${location.trim()}`,
  name: name.trim(),
  email: email.trim(),
  phone: phone.trim(),
  address: location.trim(),
  note: message || `Application to become a lister (${listerRole}) in ${location.trim()}.`,
  customData: [
    { label: "Inquiry Type", value: "Become a Lister" },
    { label: "Lister Role", value: listerRole },
    { label: "Primary Area / Location", value: location.trim() },
  ],
};
```

## 9.8 `PropertyFiltersModal.tsx` — hand-rolled filter sheet

Unlike the other modals this one is plain JSX with a `isOpen` prop, not Radix. It is always mounted and animated in/out with transforms.

```
<div className={`fixed inset-0 z-50 flex items-end sm:items-center justify-center transition-opacity duration-300 ${
    isOpen ? "opacity-100 pointer-events-auto" : "opacity-0 pointer-events-none"}`}>
  <div className={`absolute inset-0 bg-black/50 transition-opacity duration-300 ${isOpen ? "opacity-100" : "opacity-0"}`}
↪  onClick={onClose} />

  <div className={`relative w-full sm:w-[540px] max-h-[90vh] sm:max-h-[85vh] bg-white dark:bg-[#1A1A1A] rounded-t-3xl
↪  sm:rounded-3xl flex flex-col overflow-hidden shadow-2xl transition-all duration-300 ease-out transform ${
      isOpen ? "translate-y-0 opacity-100 scale-100"
             : "translate-y-full md:translate-y-10 opacity-0 scale-95"}`}>
    <div className="flex-1 overflow-auto px-6 py-5">
      ... filters ...
    </div>
  </div>
</div>
```

Location dropdowns cascade: the city dropdown is disabled until a state is chosen.

```
<CustomDropdown
  placeholder={state ? "All Cities" : "Select State first"}
  value={city}
  options={[{ label: "All Cities", value: "" }, ...cityOptions]}
  onChange={(val) => onCityChange?.(val)}
  disabled={!state}
  ...
/>
```

# 10. Inspections (book a visit)

The inspections feature is small — one modal and one store:

```
src/features/inspections/
├── components/BookInspectionModal.tsx   (636 lines)
└── store/useInspectionStore.ts          (27 lines)
```

## 10.1 `useInspectionStore.ts`

A plain Zustand store (no persistence). It is a local record of visits made in this browser session.

```ts
export const useInspectionStore = create<InspectionStore>((set) => ({
  inspections: [],
  loading: false,

  addInspection: (inspectionData) => {
    const newInspection: Inspection = {
      ...inspectionData,
      id: Math.random().toString(36).substring(7),
      status: "pending",
      paymentStatus: "pending",
    };
    set((state) => ({ inspections: [...state.inspections, newInspection] }));
  },

  updatePaymentStatus: (reference, status) => {
    set((state) => ({
      inspections: state.inspections.map((inspection) =>
        inspection.reference === reference
          ? { ...inspection, paymentStatus: status, status: status === "paid" ? "confirmed" : "pending" }
          : inspection
      ),
    }));
  },
}));
```

## 10.2 `BookInspectionModal.tsx`

Booking a visit is the same Paystack flow as buying a property, but for the visitation fee, and it also records the booking locally.

Constants:

```ts
const API_URL = import.meta.env.VITE_API_URL;
const API_KEY = import.meta.env.VITE_SABIFLOW_API_KEY;
const GATEWAY_ID = import.meta.env.VITE_SABIFLOW_GATEWAY_ID;
const INSPECTION_BOOKING_NOTICE_HOURS = 48;

// Hourly slots from 08:00 to 16:00 (stored as 24h "HH:00", displayed in 12h)
const VISIT_SLOTS = Array.from({ length: 9 }, (_, i) =>
  `${String(i + 8).padStart(2, "0")}:00`,
);
```

**Booking constraints.** The earliest bookable slot is 48 hours away, weekends are excluded, and the last slot of the day is 16:00.

```
const getVisitBookingConstraints = () => {
  const now = new Date();
  const earliestVisit = new Date(
    Math.ceil((now.getTime() + INSPECTION_BOOKING_NOTICE_HOURS * 60 * 60 * 1000) / 60000) * 60000,
  );
  const minDate = new Date(earliestVisit.getFullYear(), earliestVisit.getMonth(), earliestVisit.getDate());

  // If the 48-hour mark lands after 16:00, push to the next day
  if (earliestVisit.getHours() > 16 || (earliestVisit.getHours() === 16 && earliestVisit.getMinutes() > 0)) {
    minDate.setDate(minDate.getDate() + 1);
  }
  // Skip weekends
  while (minDate.getDay() === 0 || minDate.getDay() === 6) {
    minDate.setDate(minDate.getDate() + 1);
  }

  const minTime =
    formatLocalDate(minDate) === formatLocalDate(earliestVisit) && earliestVisit.getHours() >= 8
      ? formatTime(earliestVisit)
      : "08:00";

  return { minDate: formatLocalDate(minDate), minTime };
};
```

Slots for the selected day are filtered against the earliest allowed time, and a stale selection is cleared if it no longer qualifies:

```
const { minDate: minVisitDate, minTime: earliestVisitTime } = getVisitBookingConstraints();
const earliestTimeForDate = visitDate === minVisitDate ? earliestVisitTime : "08:00";
const [earliestHour, earliestMinute] = earliestTimeForDate.split(":").map(Number);
const firstAvailableTime = `${String(earliestHour + (earliestMinute > 0 ? 1 : 0)).padStart(2, "0")}:00`;

const availableSlots = VISIT_SLOTS.filter((t) => t >= firstAvailableTime);

React.useEffect(() => {
  if (visitTime && (visitTime < firstAvailableTime || visitTime > "16:00" || Number(visitTime.slice(3, 5)) !== 0)) {
    setVisitTime("");
  }
}, [visitTime, firstAvailableTime]);
```

Weekends are rejected in the date picker itself:

```
const isWeekendDate = (date: string) => {
  if (!date) return false;
  const day = new Date(`${date}T00:00:00`).getDay();
  return day === 0 || day === 6;
};

<input
  type="date"
  required
  min={minVisitDate}
  value={visitDate}
  onChange={(e) => {
    const selectedDate = e.target.value;
    if (isWeekendDate(selectedDate)) {
      setVisitDate("");
      setVisitTime("");
      toast.error("Please choose a weekday for your visit.");
      return;
    }
    setVisitDate(selectedDate);
  }}
/>
```

**Server-side validation** mirrors the client rules before anything is charged:

```
const selectedVisit = new Date(`${visitDate}T${visitTime}:00`);
const minimumNoticeMs = INSPECTION_BOOKING_NOTICE_HOURS * 60 * 60 * 1000;
if (
  Number.isNaN(selectedVisit.getTime()) ||
  visitTime < "08:00" ||
  visitTime > "16:00" ||
  !visitTime.endsWith(":00") ||
  selectedVisit.getTime() < Date.now() + minimumNoticeMs
) {
  toast.error("Choose a visit between 8:00 AM and 4:00 PM, at least 48 hours from now.");
  return;
}
```

**Payment flow.** Same three API calls as the property purchase, but the sale also carries the Bearer token:

```
const salePayload = {
  initiateInvoice: true,
  gatewayId: GATEWAY_ID,
  items: [
    {
      description: `Inspection Fee: ${property.name}, ${propertyAddress}`,
      quantity: 1,
      unitPrice: amount,           // property.visitationfee
      inventoryItemId: undefined,
      productId: undefined,
    }
  ],
  taxRate: 0,
  notes: `Inspection booking for ${property.name} at ${propertyAddress}.\nPreferred Visit: ${preferredVisit}`,
  paymentMethod: "credit_card",
  customerDetails: { name: fullName, email: bookingEmail, phone: phone }
};

const saleResponse = await axios.post<SaleResponse>(`${API_URL}/sales`, salePayload, {
  headers: {
    "x-api-key": API_KEY,
    Authorization: `Bearer ${token}`,
  },
});

const saleId = saleResponse.data.id || saleResponse.data._id;
```

```
const initResponse = await axios.post(`${API_URL}/sales/${saleId}/payment/initiate`, {
  gatewayId: GATEWAY_ID,
  initiateGateway: true,
}, { headers: { "x-api-key": API_KEY, Authorization: `Bearer ${token}` } });

const accessCode = initResponse.data.data.accessCode;

const paystack = new PaystackPop();
paystack.resumeTransaction(accessCode, {
  onSuccess: async (transaction: PaystackSuccessResponse) => {
    await axios.post(`${API_URL}/sales/${saleId}/payment/verify`, {
      gatewayId: GATEWAY_ID
    }, { headers: { "x-api-key": API_KEY, Authorization: `Bearer ${token}` } });
    ...
  },
  onCancel: () => { toast.info("Payment cancelled."); setIsSubmitting(false); }
});
```

**CRM + local store.** After verification the visit is pushed to the CRM and recorded locally:

```
const crmPayload = {
  companyId: COMPANY_ID,
  pipelineId: "6a4c98cbeb058fac4420ea43",
```

```
  title: `VISIT BOOKING: ${property.name} - ${fullName}`,
  name: fullName,
  amount: amount,
  email: bookingEmail,
  phone: phone,
  address: propertyAddress,
  note: `visit/inspection booking for ${property.name} at ${propertyAddress}.\nPreferred Visit: ${preferredVisit}`,
  customData: [
    { label: "Property Name", value: property.name },
    { label: "Preferred Visit", value: preferredVisit },
    { label: "Property Link", value: propertyUrl },
    { label: "Inspection Fee", value: `₦${amount.toLocaleString()}` },
    ...
  ]
};

await axios.post("https://api.sabiflow.com/api/crm/deals/guest", crmPayload);

addInspection({
  propertyId: property.id,
  propertyName: property.name,
  userName: fullName,
  userEmail: bookingEmail,
  userPhone: phone,
  date: new Date().toISOString().split('T')[0],
  time: new Date().toLocaleTimeString("en-US", { hour: "numeric", minute: "2-digit", hour12: true }),
  amount,
  reference: transaction.reference,
  location: propertyAddress,
});

updatePaymentStatus(transaction.reference, "paid");
```

Note the local record stores the *booking* date (today), while the *preferred visit* date/time lives in the CRM payload.

**The fee display** handles free inspections:

```
const amount = property.visitationfee;
const feeDisplay = amount === 0 ? "inspection is free" : `₦${amount.toLocaleString()}`;
```

# 11. Blog, Comments, FAQ and the area-map store

These four features are all **static-content** features: they read JSON files out of `public/data` and paginate client-side. None of them talk to the inventory API.

```
public/data/
├── Blog.json          (~115 KB)
├── Comments.json      (~6.6 KB)
├── FAQ.json           (~3.8 KB)
└── AreaMap.json       (~7.9 KB)
```

## 11.1 `useBlogStore.ts`

```ts
export const useBlogStore = create<BlogStore>((set, get) => ({
  posts: [],
  loading: false,
  page: 0,
  ITEMS_PER_PAGE: 6,

  fetchPosts: async () => {
    set({ loading: true });
    try {
      const res = await axios.get<BlogPost[]>("/data/Blog.json");
      const sanitizedPosts = res.data.map(post => ({
        ...post,
        image: ensureHttps(post.image)      // upgrades http:// -> https://
      }));
      set({ posts: sanitizedPosts, loading: false });
    } catch (err) {
      console.error("Failed to fetch blog posts", err);
      set({ loading: false });
    }
  },

  getPostBySlug: (slug: string) => {
    return get().posts.find((post) => post.slug === slug);
  },

  nextPage: () =>
    set((state) => ({
      page: Math.min(
        state.page + 1,
        Math.ceil(state.posts.length / state.ITEMS_PER_PAGE) - 1
      ),
    })),

  prevPage: () =>
    set((state) => ({
      page: Math.max(state.page - 1, 0),
    })),

  setPage: (page: number) => set({ page }),
}));
```

The three content stores (`useBlogStore`, `useCommentStore`, `useFAQStore`) are structurally identical — only the URL, the state key and the page size differ (`Blog` 6 per page, `Comments` 3, `FAQ` 3).

## 11.2 `BlogPage.tsx` — listing

```
const { posts, loading, page, ITEMS_PER_PAGE, fetchPosts, nextPage, prevPage } = useBlogStore();

useEffect(() => { fetchPosts(); }, [fetchPosts]);

const totalPages = Math.ceil(posts.length / ITEMS_PER_PAGE);
const currentPosts = posts.slice(page * ITEMS_PER_PAGE, (page + 1) * ITEMS_PER_PAGE);
```

Each card links to the post detail route:

```
{currentPosts.map((post) => (
  <article key={post.id} className="bg-white dark:bg-[#1A1A1A] rounded-2xl border border-gray-700/40 overflow-hidden flex
↪  flex-col hover:shadow-xl transition-shadow duration-300">
    <Link to={`/blog/${post.slug}`}>
      <img src={post.image} alt={post.title} loading="lazy" decoding="async" className="w-full h-60 object-cover" />
    </Link>
    ...
    <Link to={`/blog/${post.slug}`}>
      <h2 className="text-xl font-bold ...">{post.title}</h2>
    </Link>
    ...
  </article>
))}
```

Pagination is hidden when there is only one page:

```
{totalPages > 1 && (
  <div className="mt-12 flex justify-between items-center ...">
    <p className="text-sm text-black dark:text-white">Page {page + 1} of {totalPages}</p>
    ...
  </div>
)}
```

### 11.3 `BlogPostDetailsPage.tsx` — single post

It refetches the whole JSON then picks the post out of the store:

```
const { slug } = useParams<{ slug: string }>();
const { getPostBySlug, fetchPosts, loading, posts } = useBlogStore();
const [post, setPost] = useState<BlogPost | undefined>(undefined);

useEffect(() => {
  const loadPost = async () => {
    await fetchPosts();
    if (slug) {
      const foundPost = getPostBySlug(slug);
      setPost(foundPost);
    }
  };
  loadPost();
}, [slug, fetchPosts, getPostBySlug]);
```

Related posts = same category, excluding the current post, max 3:

```
const relatedPosts = useMemo(() => {
  if (!post) return [];
  return posts
    .filter((p) => p.category === post.category && p.id !== post.id)
    .slice(0, 3);
}, [post, posts]);
```

The body is plain text with `**bold**` markers converted to `<strong>`:

```
{post.content.split('\n').map((paragraph, index) => {
  if (!paragraph.trim()) return <div key={index} className="h-4" />;

  const parts = paragraph.split(/(\*\*.*?\*\*)/g);
  return (
    <p key={index} className="mb-4">
      {parts.map((part, i) => {
        if (part.startsWith('**') && part.endsWith('**')) {
          return <strong key={i} className="font-bold text-gray-900 dark:text-white">{part.slice(2, -2)}</strong>;
        }
        return part;
      })}
    </p>
  );
})}
```

It also emits schema.org JSON-LD:

```
<script type="application/ld+json">
  {JSON.stringify({
    "@context": "https://schema.org",
    "@type": "BlogPosting",
    "headline": post.title,
    "image": [post.image],
    "datePublished": post.date,
    "author": [{ "@type": "Organization", "name": post.author, "url": "https://rewaciti.com" }],
    "description": post.summary
  })}
</script>
```

### 11.4 `CommentSection.tsx` — testimonials

Same store pattern, but the card shows a star rating built from a string repeat:

```
const { comments, loading, page, ITEMS_PER_PAGE, fetchComments, nextPage, prevPage } = useCommentStore();

<p className="text-yellow-400 text-lg">
  {"★".repeat(comment.rating)}{"☆".repeat(5 - comment.rating)}
</p>
```

Long descriptions are truncated to 10 words with a per-card expand toggle:

```
const [expanded, setExpanded] = useState<Record<number, boolean>>({});

<p className="text-sm text-gray-600 dark:text-gray-300 mb-4">
  {expanded[comment.id] ? comment.description : truncateWords(comment.description, 10)}
  {comment.description.split(" ").length > 10 && (
    <button onClick={() => setExpanded((prev) => ({ ...prev, [comment.id]: !prev[comment.id] }))}
      className="text-[#703BF7] ml-2 hover:underline">
      {expanded[comment.id] ? "Show less" : "... Read more"}
    </button>
  )}
</p>
```

The reviewer's location links to Google Maps:

```
<a href={`https://www.google.com/maps/search/?api=1&query=${encodeURIComponent (comment.location)}`}
  target="_blank" rel="noopener noreferrer">
  {comment.location}
</a>
```

### 11.5 `AllCommentsPage.tsx` / `AllFAQPage.tsx`

Both ”View All” pages skip pagination entirely and render every item:

```
// AllCommentsPage
{loading ? (
  [...Array(6)].map((_, i) => <CommentCardSkeleton key={i} />)
) : (
  comments.map((comment: Comment) => (
    <div key={comment.id} className="rounded-xl p-5 ...">
      <p className="text-yellow-400 text-lg">{"★".repeat(comment.rating)}{"☆".repeat(5 - comment.rating)}</p>
      <h4 className="text-md font-semibold mb-2">{comment.summary}</h4>
      <p className="text-sm ...">{comment.description}</p>
      ...
    </div>
  ))
)}
```

```
// AllFAQPage
{loading ? (
  [...Array(6)].map((_, i) => <FAQCardSkeleton key={i} />)
) : (
  faq.map((item: FAQ) => (
    <div key={item.id} className="border border-gray-600/30 rounded-xl p-4 ...">
      <h4 className="text-lg font-semibold mb-2">{item.question}</h4>
      <p className="dark:text-gray-400 text-gray-600 text-sm leading-relaxed">{item.answer}</p>
    </div>
  ))
)}
```

### 11.6 `FAQSection.tsx` — home-page FAQ strip

Same as the comment section but with an 8-word truncation and an expandable answer:

```
const [expanded, setExpanded] = useState<Record<number, boolean>>({});
<p className="text-sm text-gray-600 dark:text-gray-300">
  {expanded[faq.id] ? faq.answer : truncateWords(faq.answer, 8)}
</p>
{faq.answer.split(" ").length > 8 && (
  <button onClick={() => setExpanded((prev) => ({ ...prev, [faq.id]: !prev[faq.id] }))}>
    {expanded[faq.id] ? "Show less" : " Read more"}
  </button>
)}
```

### 11.7 `useAreaMapStore.ts` — currently unused

```
export const useAreaMapStore = create<AreaMapStore>((set) => ({
  areaMaps: [],
  loading: false,
  error: null,

  fetchAreaMaps: async () => {
    set({ loading: true });
    try {
      const res = await axios.get<AreaMap[]>("/data/AreaMap.json");
      set({ areaMaps: res.data, loading: false, error: null });
    } catch (err) {
      console.error("Error fetching area maps:", err);
      set({ error: "Failed to load area map data", loading: false });
    }
  },
}));
```

This store is exported but never imported anywhere in `src/` — the map UI in the app is `PropertyMap.tsx` (§8.6), which is driven by the property store instead.

# 12. Layout: Navbar and Footer

`src/shared/components/Layout/Navbar.tsx` (519 lines) and `src/shared/components/Layout/Footer.tsx` (414 lines).

## 12.1 `Navbar.tsx`

The navbar is two bars: a small top strip that scrolls away, and a sticky main bar.

```
const { theme, toggleTheme } = useThemeStore();
const { shortlistedProperties, toggleShortlist } = usePropertyStore();
const { isAuthenticated, customer, logout } = useAuthStore();

const navItems = [
  { name: "Home", path: "/" },
  { name: "Properties", path: "/properties" },
  { name: "Services", path: "/Service" },
  { name: "Contact", path: "/Contact" },
];
```

Active-link matching treats the student-housing route as part of ”Properties”:

```
const checkActive = (path: string) => {
  const currentPath = location.pathname.toLowerCase();
  const targetPath = path.toLowerCase();

  if (targetPath === "/properties") {
    return currentPath === "/properties" || currentPath === "/studentarea";
  }

  return currentPath === targetPath;
};
```

**Avatar initials** are derived from the customer record:

```
const profileInitials = (() => {
  const firstName = customer?.firstName?.trim();
  const lastName = customer?.lastName?.trim();
  const email = customer?.email?.trim();

  if (firstName && lastName) return `${firstName[0]}${lastName[0]}`.toUpperCase();
  if (firstName) return firstName.slice(0, 2).toUpperCase();
  if (email) return email.slice(0, 2).toUpperCase();
  return "U";
})();
```

**Shortlist dropdown.** The heart icon shows a count badge and opens a panel with the shortlisted properties. It is always mounted and animated:

```
<button onClick={() => {
  if (!isAuthenticated) {
    toast.error("Please log in to access your wishlist.");
    return;
  }
  setIsShortlistOpen(!isShortlistOpen);
}}>
  <FiHeart size={18} />
  {shortlistedProperties.length > 0 && (
    <span className="absolute -top-1 -right-2 bg-[#703BF7] text-white text-[10px] font-bold rounded-full w-4 h-4 ...">
      {shortlistedProperties.length}
    </span>
```

```
  )}
</button>
```

```
<div className={`absolute right-0 top-4 mt-2 w-72 ... origin-top-right transform transition-all duration-200 ease-out ${
    isShortlistOpen ? "opacity-100 scale-100 translate-y-0 pointer-events-auto"
                    : "opacity-0 scale-95 -translate-y-2 pointer-events-none"}`}>
```

Each entry navigates to the property; the bin icon removes it:

```
{shortlistedProperties.map((property) => (
  <div key={property.id} className="p-3 hover:bg-gray-100 ... flex items-center gap-3 group">
    <img src={property.img} alt={property.name}
      onClick={() => { navigate(`/properties/${property.slug}`); setIsShortlistOpen(false); }} />
    <p className="text-xs font-semibold truncate ...">{property.name}</p>
    <button onClick={() => { toast.success("Removed from shortlist"); void toggleShortlist(property); }}>
      <FiTrash2 size={14} />
    </button>
  </div>
))}
```

**Outside-click handling** is registered for both the shortlist and the mobile menu:

```
useEffect(() => {
  const handleClickOutside = (event: MouseEvent) => {
    if (shortlistRef.current && !shortlistRef.current.contains(event.target as Node)) {
      setIsShortlistOpen(false);
    }
  };
  document.addEventListener("mousedown", handleClickOutside);
  return () => document.removeEventListener("mousedown", handleClickOutside);
}, []);
```

**Body scroll lock** while the mobile menu or the shortlist is open:

```
useEffect(() => {
  if (isOpen) document.body.style.overflow = "hidden";
  else document.body.style.overflow = "";
  return () => { document.body.style.overflow = ""; };
}, [isOpen]);
```

**Theme toggle:**

```
<button onClick={toggleTheme} className="flex items-center gap-2 px-1 py-1 rounded-full bg-gray-200 dark:bg-gray-700 ...">
  {theme === "dark" ? <FiSun size={20} /> : <FiMoon size={20} />}
</button>
```

**Auth-dependent right-hand side:** a profile avatar when signed in, otherwise a Login button.

```
{isAuthenticated ? (
  <button type="button" title="Profile" onClick={() => navigate("/auth/profile")}
    className="flex h-10 w-10 items-center justify-center rounded-full ... bg-[#703BF7] text-white">
    {profileInitials}
  </button>
) : (
  <NavLink to="/auth/login">
    {({ isActive }) => (
      <button className={isActive ? "bg-[#703BF7] text-white ..." : "bg-gray-100 ..."}>Login</button>
    )}
  </NavLink>
)}
```

**Mobile menu** slides in from the right and is also always mounted:

```
<div ref={menuRef}
  className={`fixed top-0 right-0 h-full w-64 max-w-[80%] ... z-50 flex flex-col ...
    transform transition-transform duration-300 ease-[cubic-bezier(0.32,0.72,0,1)]
    ${isOpen ? "translate-x-0" : "translate-x-full"}`}>
```

Logout lives at the bottom of the mobile menu:

```
<button onClick={() => {
  logout();
  setIsOpen(false);
  navigate("/");
  toast.success("Logged out successfully");
}} className="text-[17px] text-red-500 font-semibold ...">
  <FiLogOut size={18} /> Logout
</button>
```

The navbar also owns the ”Apply to List” modal:

```
<ListPropertyModal open={isListPropertyOpen} onOpenChange={setIsListPropertyOpen} />
```

## 12.2 `Footer.tsx`

The footer has three parts: a CTA banner, a link grid, and a bottom bar.

**Newsletter subscription** posts to the CRM:

```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();

  if (!email) {
    toast.error("Please enter your email");
    return;
  }

  setIsSubmitting(true);

  const payload = {
    companyId: COMPANY_ID,
    pipelineId: "69cec882dd40685bfe20ad43",
    title: `Newsletter Subscription: ${email}`,
    name: "Newsletter Subscriber",
    email: email,
    note: "User subscribed to newsletter via footer form.",
  };

  try {
    await axios.post("https://api.sabiflow.com/api/crm/deals/guest", payload);
    toast.success("Subscribed successfully!");
    setEmail("");
  } catch (error) {
    console.error("Error subscribing:", error);
    toast.error("Failed to subscribe. Please try again.");
  } finally {
    setIsSubmitting(false);
  }
};
```

**Link grid.** Five columns; the ”Contact Us” column is duplicated so it shows on desktop and again on mobile:

```
<div className="w-full lg:flex-1 grid grid-cols-2 sm:grid-cols-3 md:grid-cols-5 xl:gap-8 lg:gap-4 h-fit divide-x
↪  md:divide-x-0 ...">
  {/* COLUMN 1 — Home */}   <Link to="/#features">Features</Link> ...
```

```
  {/* COLUMN 2 — About Us */}<Link to="/About#Journey">Our Story</Link> ...
  {/* COLUMN 3 — Properties */}<Link to="/Properties#Categories">Categories</Link> ...
  {/* COLUMN 4 — Services */}  <Link to="/Service#whatweoffer">What We Offer</Link> ...
  {/* COLUMN 5 — Contact Us (tablet + desktop) */} ...
</div>
```

Hash links (`/#features`, `/About#Journey`, `/Service#valuation`, …) are handled by the `useScrollToHash` hook (§3).

**Bottom bar** has the copyright, terms/privacy links and social icons:

```
<span>© {currentYear} RewaCiti. All rights reserved.</span>
<Link to="/terms">Terms.</Link>
<Link to="/privacy-policy">Privacy Policy.</Link>
...
<a href="https://linkedin.com" ...><FaLinkedin /></a>
<a href="https://x.com/RewaCiti_001" ...><FaXTwitter /></a>
<a href="https://www.instagram.com/rewaciti/" ...><FaInstagram /></a>
<a href="https://www.youtube.com/@RewaCiti" ...><FaYoutube /></a>
```

# 13. Marketing and legal pages

`src/app/pages/` holds the static, content-driven pages.

| File | Route | Notes |
|---|---|---|
| `Home.tsx` | `/` | Hero + 4 feature cards + featured properties + testimonials + FAQ |
| `About.tsx` | `/About` | Static sections with hash anchors |
| `Service.tsx` | `/Service` | Static sections with hash anchors |
| `Contact.tsx` | `/Contact` | Contact form → CRM, plus offices |
| `Terms.tsx` | `/terms` | Static legal text |
| `PrivacyPolicy.tsx` | `/privacy-policy` | Static legal text |
| `NotFound.tsx` | * | 404 page |

### 13.1 `Home.tsx`

The home page composes the feature sections and gives each one an `id` so the footer links can scroll to it.

```
function Home() {
  useScrollToHash();

  const services = [
    { img: "/logo/servicecontainer/Icon Container (3).png", text: "Find Your Dream Home" },
    { img: "/logo/servicecontainer/Icon Container (4).png", text: "Unlock Property Value" },
    { img: "/logo/servicecontainer/Icon Container (5).png", text: "Effortless Property Management" },
    { img: "/logo/servicecontainer/Icon Container (6).png", text: "Smart Investment. Informed Decision" },
  ];

  return (
    <div className="">
      <Helmet>
        <title> RewaCiti | Best Real Estate platform in Ile-Ife </title>
       <meta name="description" content="RewaCiti is your number one platform for finding the best properties, rentals, and
↪  real estate services in Ile-Ife and across Osun State, Nigeria." />
        ...
      </Helmet>
      <Navbar />
      ...
```

The section ids match the footer links:

```
<section className="py-2 bg-gray-300 dark:bg-black/30" id="features"> ... </section>
<section className="bg-gray-300 dark:bg-black/30" id="properties"><PropertySection /></section>
<section className="bg-gray-300 dark:bg-black/30" id="testimonials"><CommentSection /></section>
<section className="bg-gray-300 dark:bg-black/30 pb-10" id="faq"><FAQSection /></section>
```

The hero is a two-panel layout that swaps to a stacked layout on mobile, with a floating logo in the middle:

```
<section className="relative flex flex-col sm:flex-row h-[calc(95vh-6.5rem)] sm:h-auto sm:min-h-0 overflow-hidden"
↪  id="hero">
  <div className="hidden sm:block absolute lg:left-[52%] top-1/3 md:left-[54%] left-[54%] -translate-x-1/2 -translate-y-1/2
↪  z-20">
    <img src={centerLogo} alt="Center Logo" className="w-23 h-auto" />
  </div>
  ...
```

## 13.2 `About.tsx` and `Service.tsx`

Both are pure content pages; the only logic is the scroll hook and the section ids used by the footer.

```
// About.tsx
useScrollToHash();
...
<section className="bg-gray-300 dark:bg-black/30 py-3" id="Journey"> ... </section>
<section className="bg-gray-300 dark:bg-black/30 py-3" id="Values"> ... </section>
<section className="bg-gray-300 dark:bg-black/30 py-3" id="Achievements"> ... </section>
<section className="bg-gray-300 dark:bg-black/30 py-3" id="Process"> ... </section>
<section className="bg-gray-300 dark:bg-black/30 py-3" id="Team"> ... </section>
```

```
// Service.tsx
useScrollToHash();
...
<section className="py-4 bg-gray-300 dark:bg-black/30 border-t-5 border-b-5 border-gray-600/30" id="whatweoffer"> ...
↪  </section>
<section className="bg-gray-300 dark:bg-black/30 py-4" id="valuation"> ... </section>
<section className="bg-gray-300 dark:bg-black/30 py-4" id="Management"> ... </section>
<section className="bg-gray-300 dark:bg-black/30 py-4" id="marketing"> ... </section>
```

## 13.3 `Contact.tsx` — the only marketing page with a form

It follows the same cookie + CRM pattern as the modals.

```
const inquiryCookieKey = "rewaciti_contact_inquiry";
const cookieLoaded = useRef(false);

useScrollToHash();

useEffect(() => {
  const savedInquiry = getCookie(inquiryCookieKey);
  if (savedInquiry) {
    try {
      const parsed = JSON.parse(savedInquiry);
      setFirstName(parsed.firstName || "");
      setLastName(parsed.lastName || "");
      setEmail(parsed.email || "");
      setPhone(parsed.phone || "");
      setInquiryType(parsed.inquiryType || "");
      setSource(parsed.source || "");
      setMessage(parsed.message || "");
      setAgreed(parsed.agreed || false);
    } catch {
      // ignore malformed cookie data
    }
  }

  if (!isAuthenticated || !customer) return;

  const populateProfileDetails = async () => {
    try {
      const profileData = await authAPI.getProfile();
      const latestPhone = profileData.phoneNumber || "";
      setFirstName(customer.firstName || "");
      setLastName(customer.lastName || "");
      setEmail(customer.email || "");
      setPhone(latestPhone || customer.phoneNumber || "");
      ...
    } catch (error) {
      console.error("Failed to fetch profile for contact form:", error);
    }
  };
```

```
  populateProfileDetails();
}, [isAuthenticated, customer]);
```

Submission:

```
const payload = {
  companyId: COMPANY_ID,
  pipelineId: "69cec9f3dd40685bfe20adb2",
  title: `Contact Inquiry from ${fullName}`,
  name: fullName,
  email: email,
  phone: phone,
  note: message || "No additional message added",
  customData: [
    { label: "Inquiry Type", value: inquiryType },
    { label: "Source", value: source },
  ],
};

await axios.post("https://api.sabiflow.com/api/crm/deals/guest", payload);
```

Page anchors: `Contactinfo`, `Contactform`, `Ouroffices`.

### 13.4 `Terms.tsx` / `PrivacyPolicy.tsx`

Static legal text laid out as numbered sections inside cards:

```
<div className="max-w-4xl mx-auto px-4 py-8 text-gray-800 dark:text-gray-200">
  <h1 className="text-3xl font-bold text-gray-900 dark:text-white mb-2 text-center">Terms of Use</h1>
  <p className="text-center text-gray-600 dark:text-gray-400 mb-6">Last Updated: March 2026</p>

  <section className="bg-white/90 dark:bg-[#1A1A1A] border rounded-xl p-6 mb-4">
    <h2 className="text-xl font-semibold mb-2">1. Introduction</h2>
    <p>RewaCiti is a digital real estate platform designed to connect users with verified property agents ...</p>
  </section>
  ...
```

### 13.5 `NotFound.tsx`

```
const NotFound = () => (
  <div className="bg-gray-100 dark:bg-black/30 min-h-screen flex flex-col">
    <Helmet>
      <title>404 Page Not Found | RewaCiti</title>
      ...
    </Helmet>
    <Navbar />
    <div className="flex flex-col items-center justify-center flex-1 text-center px-4 py-20">
      <h1 className="text-6xl font-bold text-gray-900 dark:text-white mb-4">404</h1>
      <h2 className="text-2xl text-gray-800 dark:text-gray-200 mb-6">Page Not Found</h2>
      <Link to="/" className="bg-[#703BF7] hover:bg-[#9677df] text-white px-6 py-3 rounded-lg transition">Go to Home</Link>
    </div>
    <Footer />
  </div>
);
```

# 14. Build, config and deployment

## 14.1 package.json

```
{
  "name": "RewaCiti",
  "linkedin": "https://www.linkedin.com/company/rewaciti",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "@babel/plugin-transform-react-jsx-source": "^7.27.1",
    "@emnapi/runtime": "^1.5.0",
    "@paystack/inline-js": "^2.22.8",
    "@radix-ui/react-dialog": "^1.1.15",
    "@radix-ui/react-navigation-menu": "^1.2.14",
    "@radix-ui/react-slot": "^1.2.4",
    "@tailwindcss/vite": "^4.1.17",
    "axios": "^1.13.2",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "leaflet": "^1.9.4",
    "lucide-react": "^0.554.0",
    "react": "18.0.0",
    "react-dom": "18.0.0",
    "react-helmet-async": "^3.0.0",
    "react-icons": "^5.5.0",
    "react-leaflet": "^4.2.1",
    "react-paystack": "^6.0.0",
    "react-player": "^3.4.0",
    "react-router": "^7.13.1",
    "sonner": "^2.0.7",
    "tailwind-merge": "^3.4.0",
    "tailwindcss": "^4.1.17",
    "zustand": "^5.0.8"
  },
  "devDependencies": {
    "@eslint/js": "^9.36.0",
    "@types/leaflet": "^1.9.22",
    "@types/node": "^24.6.0",
    "@types/paystack__inline-js": "^1.0.2",
    "@types/react": "^19.1.16",
    "@types/react-dom": "^19.1.9",
    "@vitejs/plugin-react": "^5.0.4",
    "eslint": "^9.36.0",
    "eslint-plugin-react-hooks": "^5.2.0",
    "eslint-plugin-react-refresh": "^0.4.22",
    "globals": "^16.4.0",
    "tw-animate-css": "^1.4.0",
    "typescript": "~5.9.3",
    "typescript-eslint": "^8.45.0",
    "vite": "npm:rolldown-vite@7.1.14",
    "vite-tsconfig-paths": "^5.1.4"
  },
  "overrides": {
    "vite": "npm:rolldown-vite@7.1.14",
    "picomatch": "^2.3.2",
    "react-paystack": {
      "react": "$react",
```

```
      "react-dom": "$react-dom"
    }
  }
}
```

Notes:

- Vite is pinned to **Rolldown** (`npm:rolldown-vite@7.1.14`) both as a dependency and as an `overrides` entry.
- `react-paystack` and `react-player` are installed but **never imported** — payments use `@paystack/inline-js`, and video is a plain `<video>`/`<iframe>`.
- `react` is pinned to `18.0.0` while `@types/react` is `^19.1.16`.
- `@babel/plugin-transform-react-jsx-source` and `@emnapi/runtime` are present but not referenced by any config file.

## 14.2 `vite.config.ts`

```ts
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'
import tsconfigPaths from 'vite-tsconfig-paths'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss(), tsconfigPaths()],
  server: {
    open: true,
  },
})
```

## 14.3 `tsconfig.app.json`

```json
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.app.tsbuildinfo",
    "target": "ES2022",
    "useDefineForClassFields": true,
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "types": ["vite/client"],
    "skipLibCheck": true,

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",

    /* Linting */
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedSideEffectImports": true
  },
```

```
  "include": ["src"]
}
```

`strict` + `noUnusedLocals` + `noUnusedParameters` is why the build runs `tsc -b` before Vite — unused imports fail the build.

### 14.4 `eslint.config.js`

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import tseslint from 'typescript-eslint'
import { defineConfig, globalIgnores } from 'eslint/config'
export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{ts,tsx}'],
    extends: [
      js.configs.recommended,
      tseslint.configs.recommended,
      reactHooks.configs['recommended-latest'],
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
    },
    rules: {
      '@typescript-eslint/no-unused-vars': [
        'error',
        { varsIgnorePattern: '^_', argsIgnorePattern: '^_' },
      ],
    },
  },
])
```

### 14.5 Tailwind / CSS

Tailwind v4 is driven from CSS (`@tailwindcss/vite` plugin), with a `tailwind.config.ts` kept for the `darkMode: "class"` setting:

```
import type { Config } from "tailwindcss";

export default {
  darkMode: "class",
  content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
  theme: {
    extend: {},
  },
  plugins: [],
} satisfies Config;
```

`src/app/styles/App.css`:

```
@import "tailwindcss";
@import "tw-animate-css";

@custom-variant dark (&:is(.dark *));
```

```
@theme inline {
  --radius-sm: calc(var(--radius) - 4px);
  ...
  --color-primary: var(--primary);
  ...
}
:root {
  --radius: 0.625rem;
  --background: oklch(1 0 0);
  ...
}

.dark {
  --background: oklch(0.145 0 0);
  ...
}

@layer base {
  * { @apply border-border outline-ring/50; }
  body { @apply bg-background text-foreground; }
  button, a, [role="button"] { @apply cursor-pointer; }
}
```

Dark mode is a `.dark` class on `<html>`, toggled by `useThemeStore` (§3).

Custom animation utilities used across the app:

```
@keyframes fadeInUp { from { opacity: 0; transform: translateY(20px); } to { opacity: 1; transform: translateY(0); } }
.animate-fade-in-up { animation: fadeInUp 0.4s cubic-bezier(0.16, 1, 0.3, 1) forwards; }

/* Radix Dialog overlays & contents */
.dialog-overlay-animate[data-state='open'] { animation: fadeIn 0.3s ... forwards; }
.dialog-overlay-animate[data-state='closed'] { animation: fadeOut 0.2s ... forwards; }

/* Center modal animation */
.center-modal-animate[data-state='open'] { animation: centerModalOpen 0.35s ... forwards; }
.center-modal-animate[data-state='closed'] { animation: centerModalClose 0.2s ... forwards; }
```

The `data-state` attribute is what Radix sets on `Dialog.Overlay` / `Dialog.Content`, so the open/close animations are pure CSS.

## 14.6 `index.html` — Trustpilot bootstrap

```
<head>
  <meta charset="UTF-8" />
  <link rel="icon" type="image/svg+xml" href="/Symbol.png" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta name="description" content="RewaCiti connects buyers, renters, and students in Ile-Ife with trusted real estate
  ↪  listings, housing guides, and local property services." />
  <meta property="og:title" content="RewaCiti - Real Estate & Student Housing in Ile-Ife" />
  <meta property="og:description" content="Find properties, student accommodation, and local housing guidance across
  ↪  Ile-Ife and Osun State with RewaCiti." />
  <meta property="og:image" content="https://rewaciti.com/Symbol.png" />
  <meta property="og:type" content="website" />
  <meta name="twitter:card" content="summary_large_image" />
  <meta name="twitter:title" content="RewaCiti - Real Estate & Student Housing in Ile-Ife" />
  <meta name="twitter:description" content="Find properties, student accommodation, and local housing guidance across
  ↪  Ile-Ife and Osun State with RewaCiti." />
  <meta name="twitter:image" content="https://rewaciti.com/Symbol.png" />
  <title>RewaCiti</title>
  <script>

(function(w,d,s,r,n){w.TrustpilotObject=n;w[n]=w[n]||function(){(w[n].q=w[n].q||[]).push( arguments)};
```

```
a=d.createElement(s);a.async=1;a.src=r;a.type='text/java'+s;f=d.getElementsByTagNa me(s)[0];
          f.parentNode.insertBefore(a,f)})(window,document,'script', 'https://invitejs.trustpilot.com/tp.min.js', 'tp');
          tp('register', 'V6p7jWrntHaozFCu');
  </script>
</head>
<body class ="bg-[#1E1E1E] text-white">
  <div id="root"></div>
  <script type="module" src="/src/app/main.tsx">
  </script>
</body>
```

The inline snippet defines `window.tp` **synchronously** and queues calls until the real Trustpilot script finishes loading — that is why `ServiceRatingModal` can call `window.tp?.("createInvitation", ...)` safely (§9.3).

## 14.7 `vercel.json` — SPA routing

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

Every route is rewritten to `index.html` so React Router can handle deep links like `/properties/<slug>` and `/blog/<slug>`.

## 14.8 `.env` — environment variables

```
VITE_PAYSTACK_PUBLIC_KEY=pk_test_77e00bd797baf9741ce9a4de9e81ff4c3c4675b1
VITE_API_URL=https://api.sabiflow.com/api
VITE_SABIFLOW_API_KEY=sk_live_a3dd9f88b7981065154a172f23eb4dbb826e40c21cf 8d20c8da6cf74d1444597
VITE_SABIFLOW_GATEWAY_ID=69ed740cc09e9388ba096d02
VITE_COMPANY_ID=69b4712ce95a2df514b1c789
```

Anything prefixed with `VITE_` is **inlined into the client bundle at build time** — it is not a secret. Two consequences:

- `VITE_SABIFLOW_API_KEY` is a live `sk_live_…` key shipped to every visitor. `.env` is currently **tracked by git** and `.gitignore` has no `.env` entry, so the key is in the repository history as well. It should be rotated and moved behind a server-side proxy.
- `VITE_PAYSTACK_PUBLIC_KEY` is a *test* key and is not referenced anywhere in `src/` — the Paystack inline modal is resumed with an `accessCode` returned by the backend instead.

## 14.9 Verified commands

```
npm run lint    -> clean (no output)
npm run build   -> tsc -b && vite build  (success)
```

The build prints one warning: the main chunk exceeds 500 kB (`dist/assets/index-*.js` ≈ 995 kB, ≈ 285 kB gzipped). It is a warning, not an error.

# 15. End-to-end data flows

## 15.1 Every backend endpoint the app uses

| Base URL | Endpoint | Used by |
|---|---|---|
| `https://api.sabiflow.com/api` | `POST /customers/login` | `authAPI.login` |
| | `POST /customers/register` | `authAPI.signUp` |
| | `POST /customers/verify-email` | `authAPI.verifyEmail` |
| | `POST /customers/resend-verification-otp` | `authAPI.resendOTP` |
| | `POST /customers/forgot-password` | `authAPI.forgotPassword` |
| | `POST /customers/reset-password` | `authAPI.resetPassword` |
| | `GET /customers/profile` | `authAPI.getProfile` |
| | `PUT /customers/profile` | `authAPI.updateProfile` |
| | `PUT /customers/change-password` | `authAPI.changePassword` |
| | `GET /customers/wishlist` | `usePropertyStore.fetchWishlist` |
| | `POST /customers/wishlist` | `usePropertyStore.toggleShortlist` (add) |
| | `DELETE /customers/wishlist/{id}` | `usePropertyStore.toggleShortlist` (remove) |
| | `POST /sales` | PropertyPaymentModal, BookInspectionModal |
| | `POST /sales/{id}/payment/initiate` | PropertyPaymentModal, BookInspectionModal |
| | `POST /sales/{id}/payment/verify` | PropertyPaymentModal, BookInspectionModal |
| | `GET /sales/{id}/invoice/download?format=pdf` | PropertyPaymentModal, BookInspectionModal |
| | `POST /crm/deals/guest` | every lead-capture modal, Footer, Contact page |
| `https://api.sabiflow.com/api/inventory/portal/rewacity` | `GET /products` | list, featured, search, related |
| | `GET /products/{slug}` | detail page |
| | `GET /categories` | filter bar + detail page |
| | `GET /filters` | filter options |
| local | `GET /data/Blog.json` | blog store |
| local | `GET /data/Comments.json` | comment store |
| local | `GET /data/FAQ.json` | FAQ store |
| local | `GET /data/AreaMap.json` | area-map store (unused) |
| local | `GET /data/PropertyPaymentFees.json` | `usePropertyStore.fetchPropertyFees` — **file missing from `public/data`** |

## 15.2 App boot

```
index.html  →  #root
   ↓
src/app/main.tsx  →  createRoot, StrictMode, BrowserRouter
   ↓
src/app/App.tsx   →  HelmetProvider, ThemeProvider, Toaster, Routes
   ↓
route page → Navbar + page content + Footer
```

`App.tsx` also hydrates the theme from localStorage into the `<html>` class list and restores the wishlist from the server when a token exists (§1).

## 15.3 Auth

```
Login page
  → POST /customers/login
  → token + customer
  → useAuthStore.setAuth(token, customer)
      ├─ cookie "rewaciti_auth_token"
      ├─ cookie "rewaciti_auth_customer"
      └─ Zustand persist "auth-storage"
  → profile dropdown shows initials
```

On reload, the persisted store is rehydrated and reconciled with the cookies (cookies win). `getProfile()` refreshes the customer record, and the phone number is patched back into the store when it differs.

```
Register → POST /customers/register
        → "check your email"
VerifyEmail → POST /customers/verify-email (OTP)
ForgotPassword → POST /customers/forgot-password
ResetPassword → POST /customers/reset-password
Profile page → GET/PUT /customers/profile, PUT /customers/change-password
```

## 15.4 Browsing properties

```
PropertiesPage
  → usePropertyStore.fetchProperties(params)
      → GET /inventory/portal/rewacity/products?...
      → mapSabiFlowProduct()  (normalise to the Property type)
      → set({ properties, total })
  → render PropertyCard grid  |  PropertyMap (list/map toggle)
       └─ hovering a card highlights the marker and vice versa
```

Clicking a card navigates to `/properties/:slug`:

```
PropertyDetailsPage
  → cached lookup (properties → relatedProperties → featuredProperties)
  → GET /products/{slug}          (fetchPropertyBySlug)
  → mapSabiFlowProduct()
  → render gallery, amenities, rules, bills, agent card, related properties
```

## 15.5 Wishlist / shortlist

```
Heart button (card, detail page, navbar)
  → guard: not authenticated? toast + stop
  → toggleShortlist(property)
      → POST   /customers/wishlist        (add)
      → DELETE /customers/wishlist/{id}   (remove)
      → localStorage "rewaciti_wishlist_metadata" keeps name/slug/img/location
        so the dropdown still renders if the API returns a trimmed record
```

## 15.6 Buying a property

```
PropertyPaymentModal
  1. POST /sales                      → saleId
  2. POST /sales/{saleId}/payment/initiate → accessCode
  3. new PaystackPop().resumeTransaction(accessCode)
  4. on success → POST /sales/{saleId}/payment/verify
  5. POST https://api.sabiflow.com/api/crm/deals/guest  (pipeline 69ed7a4ac09e9388ba096f1f)
  6. GET /sales/{saleId}/invoice/download?format=pdf  → auto-download receipt
  7. onPaymentSuccess → ServiceRatingModal
  8. ServiceRatingModal → POST /crm/deals/guest (pipeline 6a27f05df19944ca7f9a87dd)
                        → window.tp("createInvitation", ...)  (Trustpilot)
```

## 15.7 Booking a visit

```
BookInspectionModal
  0. validate date (weekday, ≥48h notice, 08:00–16:00 hourly slots)
  1. POST /sales                      (inspection fee, Bearer token + x-api-key)
  2. POST /sales/{saleId}/payment/initiate
  3. PaystackPop().resumeTransaction(accessCode)
  4. POST /sales/{saleId}/payment/verify
  5. POST /crm/deals/guest (pipeline 6a4c98cbeb058fac4420ea43)
  6. useInspectionStore.addInspection(...) + updatePaymentStatus(ref, "paid")
  7. download receipt
```

## 15.8 Lead capture (CRM)

Every form below ends in the same call, differing only by `pipelineId` and `customData`:

| Form | File | Pipeline |
|---|---|---|
| Contact page | `src/app/pages/Contact.tsx` | `69cec9f3dd40685bfe20adb2` |
| Newsletter | `src/shared/components/Layout/Footer.tsx` | `69cec882dd40685bfe20ad43` |
| Property inquiry | `InquiryModal.tsx` | `69b49c7541d35d158e336621` |
| Property preference | `PreferenceModal.tsx` | `69b49c7541d35d158e336621` |
| Book a visit | `BookInspectionModal.tsx` | `6a4c98cbeb058fac4420ea43` |
| Service rating | `ServiceRatingModal.tsx` | `6a27f05df19944ca7f9a87dd` |
| Property payment | `PropertyPaymentModal.tsx` | `69ed7a4ac09e9388ba096f1f` |
| Become a lister | `ListPropertyModal.tsx` | `6a9eb52b21202229fbb9eb00` |
| Co-rent request | `CoRentModal.tsx` | `6a9a70d121202229fbb92a98` |
| Report an agent | `ReportAgentModal.tsx` | `6a2a0b1ff19944ca7f9aa4b9` |

## 15.9 Static content

```
Home / BlogPage / BlogPostDetailsPage → /data/Blog.json
Home / CommentSection / AllCommentsPage → /data/Comments.json
Home / FAQSection / AllFAQPage → /data/FAQ.json
```

All three stores fetch once on mount and paginate client-side; the ”View All” pages render the full set.

## 15.10 Where state lives

| State | Location | Persisted? |
|---|---|---|
| Theme (light/dark) | `useThemeStore` | localStorage + `<html class="dark">` |
| Auth token + customer | `useAuthStore` | cookies + Zustand persist (`auth-storage`) |
| Property list / filters / pagination | `usePropertyStore` | in memory (reset on reload) |
| Wishlist | `usePropertyStore` | server-side + localStorage metadata cache |
| Inspections | `useInspectionStore` | in memory only |
| Blog / comments / FAQ | their stores | in memory (data from static JSON) |
| Form drafts | per-modal cookies | cookie (`rewaciti_*`) |

*End of document.*